

基于热质耦合的药材烘干模型

摘要

药材烘干过程同时涉及内部的热传导、水分扩散和表面的对流换热、传质，并伴随材料物性及几何尺寸的变化。如何准确描述药材内部温度与水分浓度的演化，进而确定满足要求的烘干时长，是优化烘干工艺、降低能耗并保证产品品质的关键。本文将药材视为细长圆柱体，忽略轴向梯度，建立一维径向热质传递模型；然后采用有限体积法完成空间离散，将偏微分方程组降维成常微分方程组，并利用后向差分公式（BDF）进行自适应时间积分。在此基础上，依次研究恒定物理性质、非线性热质耦合、烘干终止判定以及半径动态收缩条件下的数值求解问题。

针对问题一 首先根据圆柱药材长度较长的特征，将温度和水分浓度近似为仅随径向位置和时间变化的函数，建立线性热传导方程与扩散系数随含水率变化的非线性水分扩散方程。在圆心处设置对称边界，并在表面设置对流换热、传质的 Robin 边界条件。利用附件中的烘房温度和水分浓度数据构造分段线性插值函数，用有限体积法把半径离散，再通过 BDF 求解离散后的常微分方程组，得到 $0 \sim 1800\text{ s}$ 内药材各径向位置的温度与水分浓度变化规律。计算表明，预热过程中药材表面温度上升快于中心温度，水分浓度呈现中心高、表面低的径向分布。在 $t = 1800\text{ s}$ 时，对应 $r = (0, 0.5, 1.0, 1.5, 2.0)\text{ cm}$ 处的计算结果为：温度向量 $T_{1800} = (33.5754, 33.7721, 34.3641, 35.3622, 36.7859)^\circ\text{C}$ ；含水率向量 $C_{1800} = (2.5500, 2.5497, 2.5383, 2.3755, 1.5103)\text{ kg/kg}$ 。

针对问题二 在问题一的基础上，建立参数随温度、含水率变化的非线性热质耦合模型。对相邻节点间的导热系数和扩散系数采用调和平均，从而保证跨界面通量连续；将烘房环境划分为动态预热段与稳定恒温段，前段用附件数据插值，后段近似取 $T_a^* = 50^\circ\text{C}$ 、 $C_a^* = 0.05\text{ kg/kg}$ 。采用 $\Delta r = 0.05\text{ cm}$ 的细网格求解，并进行时空插值和结果输出，最终得到前 3 h 内药材温度与水分浓度的演化曲线。当 $t = 3\text{ h}$ 时，中心与表面的计算结果为： $T_c = 49.8484^\circ\text{C}$ ， $T_s = 49.8975^\circ\text{C}$ ； $C_c = 1.7658\text{ kg/kg}$ ， $C_s = 1.0080\text{ kg/kg}$ 。

针对问题三 在问题二基础之上，将烘干完成条件定义为全场最大含水率低于目标阈值，即 $t_{\text{dry}} = \inf\{t > 0 : \max_{0 \leq r \leq R_0} C(r, t) < 0.15\text{ kg/kg}\}$ 。在 BDF 求解器中设置终止事件，并将仿真时间延长至所有径向节点均满足阈值。数值结果显示，药材中心的水分浓度始终最大，是决定烘干时长的控制位置。连续阈值时刻为 207246.43 s (57.5685 h)；取首个严格达标的 60 s 输出时刻，最终烘干时间为 207300 s (57.5833 h)。此时各指定位置的水分浓度为 $C_{\text{dry}} = (0.1500, 0.1477, 0.1402, 0.1250, 0.0525)\text{ kg/kg}$ ，依次对应 $r = (0, 0.5, 1.0, 1.5, 2.0)\text{ cm}$ 。未舍入的最大值为 0.14998444 kg/kg ，满足严格阈值条件。

针对问题四 该问题需同时考虑药材失水引起的径向收缩。依据附件半径数据构造 $R(t)$ 的分段线性插值函数，并引入无量纲坐标 $\xi = r/R(t)$ ($0 \leq \xi \leq 1$)，将随时间变化的物理区域映射至固定计算区间。坐标变换后，在热传导与水分扩散方程中加入网格运动项 $\xi \dot{R}(t)/R(t)$ 及尺度因子 $1/R^2(t)$ ，结合附录四给出的物性关系，在固定 ξ 网格上继续采用有限体积法与 BDF 求解，并以 $\max_i C_i < 0.15\text{ kg/kg}$ 作为终止条件。为区分物性变化与几何收缩的影响，分别计算“固定半径-附录三物性”“固定半径-附录四物性”和“动态半径-附录四物性”三组方案。完整收缩模型的主要结果为：连续理论达标时刻 $t_{\text{con}} = 189768.43\text{ s}$ (52.7135 h)，安全输出时刻 $t_{\text{dry}} = 189780\text{ s}$ (52.7167 h)，此时药材半径收缩至 $R(t_{\text{dry}}) = 1.2000\text{ cm}$ ，最大含水率 $\max_i C_i = 0.14999407\text{ kg/kg}$ 。与问题三相比，烘干时长缩短 17520 s (4.8667 h)，降幅为 8.45% ；与采用附录四物性但保持固定半径的对照模型相比，烘干时长缩短 77.3000 h ，降幅为 59.45% ，干燥速度提高至原来的 2.466 倍。结果说明，径向收缩通过缩短水分扩散距离显著提高了烘干效率。

最后给出了建模过程的误差分析和结果检验，本文结论正确合理，具有一定的应用价值和普适性。

关键词：热质耦合；有限体积法；BDF；非线性扩散；移动边界；药材烘干

1 问题重述

1.1 问题背景

干燥是中药材加工过程中影响成品品质的重要工序。该工艺通常经历预热平衡和恒温干燥两个阶段：烘房通过调节温度与水分浓度，使热量由药材表面逐渐向内部传递，同时促使内部水分向外扩散并由热风带走。药材内部温度场和水分浓度场的变化相互关联，其演化过程还会受到材料物性、外部环境以及药材尺寸变化等因素的共同影响。

在实际生产中，若烘干参数设置不合理，容易出现干燥时间过长、能源利用率偏低以及内外干燥程度不一致等问题，进而影响药材质量。传统的试验优化方法需要反复调整工艺，不仅成本较高，而且周期较长。因此，有必要根据传热传质机理建立数学模型，借助数值计算刻画药材内部温度和水分浓度的变化规律，并进一步预测烘干完成时间，为烘干工艺的分析与优化提供依据。

1.2 题目重述

现有某种中药材，其外形可近似看作长为 25 cm、半径为 2 cm 的圆柱体。烘干开始时，药材内部温度均为 28 °C，水分浓度（干基含水率）均为 2.55 kg/kg。烘房在烘干初期的温度和水分浓度随时间变化，药材的密度、比热容、热传导系数及水分扩散系数在不同阶段采用不同的经验关系；与此同时，药材还会因水分流失而产生径向收缩。现需根据题目给出的环境数据、半径数据及相关物性参数，研究以下四个问题。

问题一：针对预热平衡阶段，分析外部烘房环境变化对圆柱药材内部传热、传质过程的影响，建立药材温度和水分浓度随时间及径向位置变化的数学模型。计算 0 ~ 1800 s 内药材内部的温度场和水分浓度场，给出题目指定时刻及距中心 0、0.5、1、1.5、2 cm 处的结果，并按规定的时间、空间间隔保存完整数据。

问题二：考虑烘干过程一般持续 2 ~ 3 天，且预热平衡阶段与恒温干燥阶段的环境和物性特征存在差异。采用题目给出的统一经验公式，建立能够描述整个烘干过程的温度-水分耦合模型，求出前 3 h 内药材各处温度和水分浓度的变化情况，并输出指定时刻、指定径向位置及完整时空网格上的计算结果。

问题三：在问题二所建立的全过程模型基础上，以药材内部所有位置的水分浓度均低于 0.15 kg/kg 作为烘干完成标准，确定满足该条件所需要的烘干时长。给出烘干过程中每隔 6 h、距中心每隔 0.5 cm 的水分浓度，并按题目要求保存全过程的水分浓度数据。

问题四：进一步考虑药材在失水过程中发生的尺寸收缩，根据附件所给半径随时间的变化数据以及新的物性经验公式，对问题三模型进行修正。在变化区域内重新计算药材水分浓度的时空分布，确定考虑尺寸变化后的烘干时长，并给出指定时刻、指定位置及药材表面处的水分浓度，同时形成规定格式的完整结果文件。

2 问题分析

药材烘干本质上是热量传递与水分迁移同步进行的非稳态过程，二者具有明显的**热质相似性**。热传导遵循傅里叶定律，水分扩散遵循菲克定律；对于轴对称圆柱药材，其控制方程均可写成“局部积累量等于径向净通量”的形式：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \quad \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right). \quad (1)$$

其中，温度 T 与水分浓度 C 、热传导系数 k 与水分扩散系数 D 、表面对流换热系数 h 与传质系数 h_m 分别构成对应关系。两类场在圆柱中心均满足对称条件，在药材表面均满足第三类边界条件。因此，可在统一的径向控制体积网格上离散温度场与水分场，再通过材料物性随 C 、 T 的变化体现二者之间的耦合。需要注意的是，热量由高温处向低温处传递，而水分由高浓度处向低浓度处扩散，二者的量纲、扩散速率及外部驱动力并不相同，故只能共享建模框架，不能将两种传递过程简单等同。

基于上述相似性，本文将药材视为长径比较大的圆柱体，忽略轴向与周向差异，重点研究温度和水分浓度沿半径方向的变化。空间上采用有限体积法离散，使每个控制体均保持能量与质量守恒；时间上将偏微分方程组转化为常微分方程组，并采用适用于刚性系统的 BDF 方法求解。四个问题由“固定半径下的短时预热”逐步推广至“物性变化的全过程烘干”和“考虑收缩的移动边界烘干”，前一问所得模型为后一问提供基础。

2.1 问题一分析

问题一要求在给定烘房温度和水分浓度变化条件下，计算预热平衡阶段药材内部温度场与水分浓度场，并给出指定时刻和径向位置上的结果。该问题的关键在于：外界环境随时间变化，水分扩散系数又随局部含水率改变，因而即使材料的热学参数取常数，整个传热传质过程仍具有时变边界和非线性扩散的特征。

首先，根据药材几何尺寸及长圆柱假设，分别建立一维径向热传导方程和水分扩散方程；将附件给出的烘房温度、烘房水分浓度数据作分段线性插值，作为药材表面的动态边界条件。圆柱中心处由于 $r = 0$ ，控制方程含有形式上的奇点，应利用中心控制体的守恒关系进行单独离散；药材表面则分别建立对流换热与对流传质条件。

其次，在同一径向网格上利用有限体积法计算各控制体之间的热通量与水分通量，得到药材各离散位置温度和水分浓度随时间变化的方程组。由于扩散系数 $D(C)$ 随求解结果实时变化，采用隐式 BDF 方法进行时间积分，以提高非线性刚性系统长时间计算的稳定性。最后，利用数值解的连续输出功能读取题目指定时刻和位置的结果，并按规定的时间、空间间隔形成完整数据文件。为排除网格尺寸对结果的显著影响，还需比较网格加密前后的最大误差，验证离散结果已经基本稳定。

2.2 问题二分析

问题二将计算范围由 1800s 的预热平衡阶段扩展到持续 2 ~ 3 天的完整烘干过程。与问题一相比，药材的密度、比热容、热传导系数均随水分浓度改变，水分扩散系数同时受温度和水分浓度影响。因此，该问题不再是两个相互独立的扩散方程，而是物性参数不断更新的非线性热质耦合问题。

在问题一模型的基础上，采用附件三给出的 $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 与 $D(C, T)$ 经验关系替换常参数，并在每一个时间步根据当前温度场和水分场更新材料物性。相邻控制体界面处的热传导系数和水分扩散系数采用调和平均，以保证通量连续并减小物性突变引起的数值误差。烘房环境则按实际工艺划分为预热变化阶段和随后近似稳定的恒温干燥阶段。

空间离散后，温度和水分浓度共同组成耦合常微分方程组。由于全过程时间尺度较长，而早期温度变化与后期水分缓慢迁移的时间尺度差异明显，仍采用具有自适应步长的 BDF 方法进行求解。和问题一相比，无需重新构造数值框架，只需更新材料物性与外界条件，即可得到前 3h 及完整烘干时段内的温度、水分浓度分布。

2.3 问题三分析

问题三要求确定药材内部所有位置的水分浓度均低于 0.15 kg/kg 时的烘干时长。该问实质上不是建立新的控制方程，而是在问题二全过程耦合模型上增加终止判据，将定时长仿真转化为未知终止时刻的事件求解问题。

由于药材表面直接与干燥空气接触，通常比内部更早达到目标含水率，靠近中心的位置可能成为决定烘干完成时间的关键。但为避免仅考察中心点造成误判，应以全部离散位置中的最大水分浓度作为判定指标，即构造

$$g(t) = \max_i C_i(t) - 0.15. \quad (2)$$

当 $g(t)$ 首次由正值降至零时，说明所有位置均已满足干燥标准，此时对应的时刻即为所求烘干时长。计算时继续沿用问题二的有限体积离散与 BDF 积分，并启用终止事件自动捕捉临界时刻；随后利用稠密输出提取每隔 6h、各指定半径处的水分浓度，从而同时给出烘干时长和全过程分布结果。

2.4 问题四分析

问题四进一步考虑药材失水收缩，使计算区域的半径随时间变化。此时水分扩散不仅取决于局部浓度梯度，还受到边界移动以及扩散距离缩短的影响。因此，该问题是在问题三基础上的移动边界非线性扩散问题，其难点在于原有固定径向网格不能直接适用于不断变化的物理区域。

首先, 根据附件二的半径数据构造分段线性函数 $R(t)$, 并引入无量纲坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \quad (3)$$

把随时间收缩的物理区域映射到固定计算区间。坐标变换后, 控制方程中除原有的扩散项外, 还会出现由 $\dot{R}(t)$ 引起的网格运动项; 表面传质边界中的空间尺度也应随 $R(t)$ 同步修正。随后采用附件四给出的新物性公式, 在固定的 ξ 网格上重新进行有限体积离散, 并继续使用 BDF 方法推进时间。

最后, 以变半径模型中 $\max_i C_i(t) = 0.15$ 作为烘干终止条件, 求得考虑尺寸变化后的烘干时长。输出时将无量纲位置换算为各时刻的实际半径, 同时单独保留药材表面节点的数据, 以满足题目对指定时刻、指定位置及表面含水率的要求。将本问结果与问题三对比, 还可分析径向收缩对扩散距离、内部水分迁移速度和总烘干时间的影响。

3 模型假设

1. 假设药材为半径 2 cm、长度 25 cm 的规则圆柱体, 材料连续、均匀且各向同性, 内部无裂纹、孔洞及热源或水分源。
2. 假设药材的初始温度和初始含水率均匀, 且外界热风沿圆周方向均匀分布, 因此温度场和含水率场均具有轴对称性。
3. 假设药材内部仅考虑 Fourier 热传导和 Fick 有效水分扩散, 表面仅考虑与环境之间的对流换热和对流传质; 忽略热辐射、内部空气对流及压力驱动渗流。
4. 假设各阶段环境温度、相对湿度以及表面对流换热、传质系数在给定时段内保持稳定, 药材表面不存在额外接触热阻。
5. 在不考虑收缩的阶段, 假设药材几何尺寸保持不变; 考虑收缩时, 假设截面始终保持圆形并沿径向均匀收缩, 忽略翘曲、开裂及各向异性变形。

4 符号说明

表 1: 主要符号说明

符号	含义	单位
r	到药材中心的距离	m
R	药材半径	m
ξ	相对坐标	—
T	药材内部温度	K
C	药材干基含水率	kg/kg
T_a	烘房环境温度	K
C_a	烘房环境水分浓度	kg/kg
ρ	药材表观密度	kg/m ³
c_p	药材定压比热容	J/(kg · K)
k	药材导热系数	W/(m · K)
D	水分有效扩散系数	m ² /s
h	表面对流换热系数	W/(m ² · K)
h_m	表面对流传质系数	m/s
N	径向网格划分区间数	—
Δr	物理空间网格步长	m
$\Delta \xi$	相对坐标空间步长	—
Δt	时间推进步长	s
ω_i^r	物理控制体几何权重	m ²
ω_i^ξ	相对坐标控制体几何权重	—
Γ	广义界面传递系数	W/(m · K) 或 m ² /s
t_d	干燥达标理论临界时刻	h
t_{safe}	60 s 网络安全达标时刻	h

5 模型建立

药材的热风烘干过程本质是热量由烘房向药材内传导、水分由药材内向表面迁移并蒸发两个过程的耦合。热量扩散的过程称为传热，水分扩散的过程称为传质，两者具有很强的相似性。传热满足 Fourier 导热定律和能量守恒；传质的过程满足 Fick 定律和质量守恒。

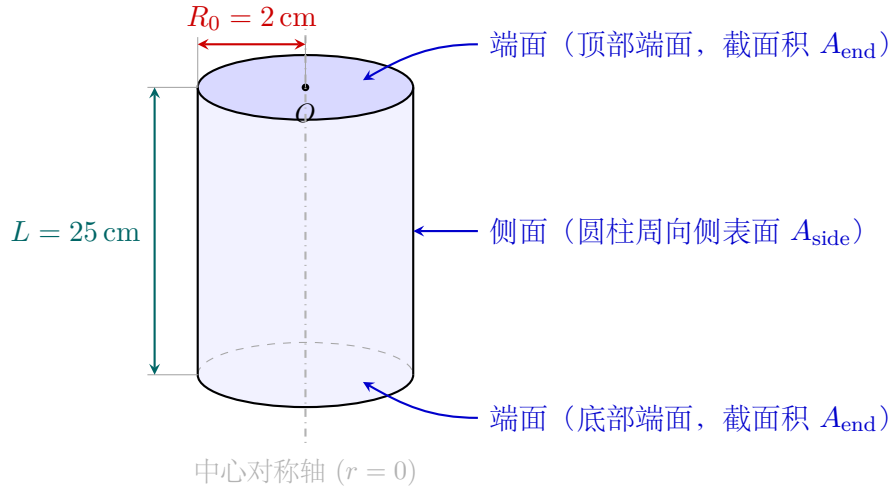


图 1: 圆柱药材几何结构及侧面、端面划分示意图

5.1 直角坐标系下的扩散方程和守恒定律

传热和传质都是扩散过程, 分别满足 Fourier 定律和 Fick 定律分别如下面(4), (5)所示:

$$\mathbf{q} = -k\nabla T, \quad (4)$$

$$\mathbf{J} = -D\nabla C, \quad (5)$$

其中 $\mathbf{q}$ 为热流密度矢量, $\mathbf{J}$ 为水分扩散通量矢量, k 为导热系数, D 为水分的扩散系数, T 为绝对温度, C 为干基含水率。

其次, 温度和含水量还分别满足能量守恒和质量守恒:

$$\rho c_p \frac{\partial T}{\partial t} = -\nabla \cdot \mathbf{q}, \quad (6)$$

$$\frac{\partial C}{\partial t} = -\nabla \cdot \mathbf{J}, \quad (7)$$

其中 ρ 为药材密度, c_p 为定压比热容。

分别联立(4)(6)和(5)(7)得到:

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k\nabla T), \quad (8)$$

$$\frac{\partial C}{\partial t} = \nabla \cdot (D\nabla C). \quad (9)$$

在三维直角坐标系 (x, y, z) 下展开上述方程，得到三维热质耦合方程如下：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(k \frac{\partial T}{\partial x} \right) + \frac{\partial}{\partial y} \left(k \frac{\partial T}{\partial y} \right) + \frac{\partial}{\partial z} \left(k \frac{\partial T}{\partial z} \right), \quad (10)$$

$$\frac{\partial C}{\partial t} = \frac{\partial}{\partial x} \left(D \frac{\partial C}{\partial x} \right) + \frac{\partial}{\partial y} \left(D \frac{\partial C}{\partial y} \right) + \frac{\partial}{\partial z} \left(D \frac{\partial C}{\partial z} \right). \quad (11)$$

5.2 柱坐标变换

由于假定中药材外观呈规则圆柱体，自然想到使用柱坐标来简化方程。

直角坐标与柱坐标的正逆变换关系为：

$$\begin{cases} x = r \cos \theta, \\ y = r \sin \theta, \\ z = z, \end{cases} \iff \begin{cases} r = \sqrt{x^2 + y^2}, \\ \theta = \arctan(y/x), \\ z = z. \end{cases} \quad (12)$$

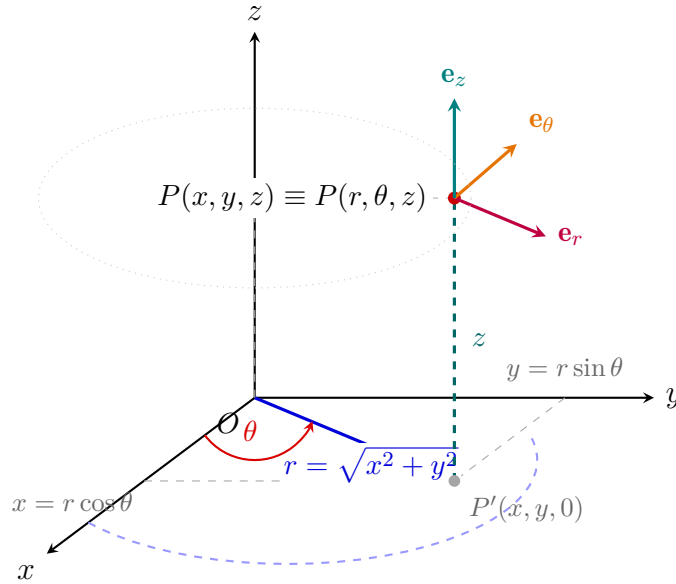


图 2: 直角坐标系 (x, y, z) 与正交柱坐标系 (r, θ, z) 的几何变换示意图

根据多元微积分复合函数的链式求导法则，两种坐标系下的偏微分算子满足如下关系：

$$\begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{\partial r}{\partial x} & \frac{\partial \theta}{\partial x} & 0 \\ \frac{\partial r}{\partial y} & \frac{\partial \theta}{\partial y} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial r} \\ \frac{\partial}{\partial \theta} \\ \frac{\partial}{\partial z} \end{bmatrix} = \begin{bmatrix} \cos \theta & -\frac{\sin \theta}{r} & 0 \\ \sin \theta & \frac{\cos \theta}{r} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial r} \\ \frac{\partial}{\partial \theta} \\ \frac{\partial}{\partial z} \end{bmatrix}. \quad (13)$$

结合度规系数可知柱坐标下的 ∇ 可以表示为：

$$\begin{aligned}\nabla &= \frac{1}{h_r} \frac{\partial}{\partial r} \mathbf{e}_r + \frac{1}{h_\theta} \frac{\partial}{\partial \theta} \mathbf{e}_\theta + \frac{1}{h_z} \frac{\partial}{\partial z} \mathbf{e}_z \\ &= \mathbf{e}_r \frac{\partial}{\partial r} + \mathbf{e}_\theta \frac{1}{r} \frac{\partial}{\partial \theta} + \mathbf{e}_z \frac{\partial}{\partial z}.\end{aligned}\quad (14)$$

对于矢量场 $\mathbf{A}$ 有柱坐标下的散度公式：

$$\begin{aligned}\nabla \cdot \mathbf{A} &= \frac{1}{h_r h_\theta h_z} \left[\frac{\partial}{\partial r} (h_\theta h_z A_r) + \frac{\partial}{\partial \theta} (h_r h_z A_\theta) + \frac{\partial}{\partial z} (h_r h_\theta A_z) \right] \\ &= \frac{1}{r} \frac{\partial}{\partial r} (r A_r) + \frac{1}{r} \frac{\partial A_\theta}{\partial \theta} + \frac{\partial A_z}{\partial z}.\end{aligned}\quad (15)$$

使用(14)(15)展开(8)和(9)得到：

$$\nabla \cdot (k \nabla T) = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) + \frac{1}{r^2} \frac{\partial}{\partial \theta} \left(k \frac{\partial T}{\partial \theta} \right) + \frac{\partial}{\partial z} \left(k \frac{\partial T}{\partial z} \right), \quad (16)$$

$$\nabla \cdot (D \nabla C) = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) + \frac{1}{r^2} \frac{\partial}{\partial \theta} \left(D \frac{\partial C}{\partial \theta} \right) + \frac{\partial}{\partial z} \left(D \frac{\partial C}{\partial z} \right). \quad (17)$$

得到柱坐标下的热质耦合方程如下：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) + \frac{1}{r^2} \frac{\partial}{\partial \theta} \left(k \frac{\partial T}{\partial \theta} \right) + \frac{\partial}{\partial z} \left(k \frac{\partial T}{\partial z} \right), \quad (18)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) + \frac{1}{r^2} \frac{\partial}{\partial \theta} \left(D \frac{\partial C}{\partial \theta} \right) + \frac{\partial}{\partial z} \left(D \frac{\partial C}{\partial z} \right). \quad (19)$$

5.3 柱坐标扩散方程的简化

式(18)与式(19)给出了柱坐标下的热质耦合方程。现在要对其进行近似简化，理由如下：

- (1) **旋转对称性**：已经假设药材外观呈规则圆柱体，并且质地均匀，具备严格的几何旋转对称性，以及烘房内热风流动平稳且均匀掠过圆柱外表面。环境温度 $T_a(t)$ 、环境水分浓度 $C_a(t)$ 以及表面对流换热与传质系数 h, h_m 均不依赖于周向角 θ 。因此，温度场和水分场满足周向各向同性，即 $\frac{\partial T}{\partial \theta} = 0$ 、 $\frac{\partial C}{\partial \theta} = 0$ 。
- (2) **大长径比与大面积比**：药材实测几何尺寸为长度 $L = 25 \text{ cm}$ ，初始半径 $R_0 = 2 \text{ cm}$ （直径 $2R_0 = 4 \text{ cm}$ ），长径比为 $L/(2R_0) = 6.25 \gg 1$ 。圆柱周向侧面积为 $A_{\text{side}} = 2\pi R_0 L$ ，而两端截面积之和仅为 $2A_{\text{end}} = 2\pi R_0^2$ 。侧面积与端面积之比高达 $A_{\text{side}}/(2A_{\text{end}}) = L/R_0 = 12.5$ ，这意味着超过 92.6% 的热量吸收与水分蒸发由侧表面承担，两端面通量占比极低，端部效应仅局限在极小的区域。
- (3) **特征传递尺度与时间尺度**：热量由外周传入中心、水分的径向特征距离仅为半径 $R_0 = 2 \text{ cm}$ ；而沿轴向传递的特征距离为半长 $L/2 = 12.5 \text{ cm}$ 。由扩散的特征时间尺

度准则 $\tau \sim l^2/D_{\text{eff}}$ 可知, 轴向传递与径向传递特征时间之比为 $\tau_z/\tau_r \approx (L/2R_0)^2 = 6.25^2 \approx 39$ 倍。表明径向传递的速度远高于轴向, 整个烘干过程中径向热质扩散占据绝对主导。

(4) **Biot 数准则**: 分别计算热传导与水分扩散的 Biot 数:

$$Bi_T = \frac{hR_0}{k} \approx \frac{25 \times 0.02}{0.36} \approx 1.39 > 0.1, \quad (20)$$

$$Bi_m = \frac{h_m R_0}{D} \approx \frac{8.0 \times 10^{-7} \times 0.02}{10^{-9}} \sim 16 \gg 0.1. \quad (21)$$

在传热学与传质学准则中, 当 $Bi > 0.1$ 时, 介质内部的导热阻力与水分扩散阻力不可忽略, 系统不能视为均质简化的集总参数模型, 药材内部必定长期维持显著的径向温度梯度与水分梯度, 即 $\frac{\partial T}{\partial r} \neq 0$ 、 $\frac{\partial C}{\partial r} \neq 0$ 。

小结 依据上述四条证据, 可以将三维柱坐标方程降维为一维, 将证据单列如下:

(1) **旋转对称性**:

$$\frac{\partial T}{\partial \theta} = 0, \quad \frac{\partial C}{\partial \theta} = 0. \quad (22)$$

(2) **大长径比与大面积比**:

$$\left| \frac{\partial T}{\partial z} \right| \ll \left| \frac{\partial T}{\partial r} \right| \Rightarrow \frac{\partial T}{\partial z} \approx 0, \quad \left| \frac{\partial C}{\partial z} \right| \ll \left| \frac{\partial C}{\partial r} \right| \Rightarrow \frac{\partial C}{\partial z} \approx 0. \quad (23)$$

(3) **Biot 数**:

$$Bi_T > 0.1 \Rightarrow \frac{\partial T}{\partial r} \neq 0, \quad Bi_m \gg 0.1 \Rightarrow \frac{\partial C}{\partial r} \neq 0. \quad (24)$$

简化后的扩散方程 于是, 三维柱坐标控制方程(18)与(19)严格退化为一维径向非线性热质耦合方程。问题 1-3 在固定物理半径 $0 \leq r \leq R$ 上满足:

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rk(C)\frac{\partial T}{\partial r}\right), \quad (25)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rD(C,T)\frac{\partial C}{\partial r}\right). \quad (26)$$

5.4 边界条件

为了求解这个偏微分方程, 还需要确定几组边界条件:

(1) **圆柱中心对称条件**: 在药材中心轴线 $r = 0$ 处, 由于具备中心对称性, 温度场与水分场关于轴心线呈镜像对称, 于是径向梯度必恒等于零:

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0. \quad (27)$$

(2) **外表面 Robin 传热与传质条件**：在药材与烘房热风接触的侧表面 $r = R$ 处，传热与传质遵循界面通量守恒原理。药材由内向外的热通量与水分通量，等于表面向外部环境对流换热与对流蒸发传质的净通量，分别服从 Newton 冷却定律与表面对流传质定律：

$$-k(C_s) \frac{\partial T}{\partial r} \Big|_{r=R} = h [T(R, t) - T_a(t)], \quad (28)$$

$$-D(C_s, T_s) \frac{\partial C}{\partial r} \Big|_{r=R} = h_m [C(R, t) - C_a(t)], \quad (29)$$

其中 $T_s = T(R, t)$ 与 $C_s = C(R, t)$ 分别表示药材外表面的温度与干基含水率， $T_a(t)$ 与 $C_a(t)$ 分别为由附件 1 给出的时变烘房空气温度与水分浓度。

总结 综合简化后的扩散方程与边界定解条件，问题 1-3 在 $0 \leq r \leq R$ 下构成的热质耦合方程组可统一表示为：

$$\left\{ \begin{array}{ll} \text{传热方程:} & \rho(C)c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(rk(C) \frac{\partial T}{\partial r} \right), \\ \text{传质方程:} & \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(rD(C, T) \frac{\partial C}{\partial r} \right), \\ \text{中心对称条件:} & \frac{\partial T}{\partial r} \Big|_{r=0} = \frac{\partial C}{\partial r} \Big|_{r=0} = 0, \\ \text{表面对流换热:} & -k(C_s) \frac{\partial T}{\partial r} \Big|_{r=R} = h [T(R, t) - T_a(t)], \\ \text{表面对流传质:} & -D(C_s, T_s) \frac{\partial C}{\partial r} \Big|_{r=R} = h_m [C(R, t) - C_a(t)]. \end{array} \right. \quad (30)$$

针对具体求解，将式(30)分别与各问对应附录中的物性参数联立即可。

6 问题求解

6.1 问题 1：恒定热物性下的预热阶段温度与水分场

6.1.1 模型建立

问题 1 研究前 1800 s 预热平衡阶段内药材温度与水分浓度的瞬态变化。该阶段药材半径保持为 $R = 0.02 \text{ m}$ ，附录 2 给出的密度、比热容和热传导系数均取常数，而水分

扩散系数随局部干基含水率变化，即

$$\begin{aligned} \rho_1 &= 820 \text{ kg/m}^3, & c_{p1} &= 2600 \text{ J/(kg} \cdot \text{K)}, \\ k_1 &= 0.36 \text{ W/(m} \cdot \text{K)}, & D_1(C) &= 7.0 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{ m}^2/\text{s}. \end{aligned} \quad (31)$$

将上述物性参数及附件 1 给出的烘房环境边界代入第五章建立的一维径向模型，得到问题 1 的完整初边值问题：

$$\left\{ \begin{array}{ll} \text{传热方程:} & \rho_1 c_{p1} \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k_1 \frac{\partial T}{\partial r} \right), \\ \text{传质方程:} & \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D_1(C) \frac{\partial C}{\partial r} \right), \\ \text{中心对称条件:} & \left. \frac{\partial T}{\partial r} \right|_{r=0} = \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0, \\ \text{表面对流换热:} & -k_1 \left. \frac{\partial T}{\partial r} \right|_{r=R} = h[T(R, t) - T_a(t)], \\ \text{表面对流传质:} & -D_1(C_s) \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m[C(R, t) - C_a(t)], \\ \text{初始条件:} & T(r, 0) = 301.15 \text{ K}, \quad C(r, 0) = 2.55. \end{array} \right. \quad (32)$$

其中 $h = 25 \text{ W/(m}^2 \cdot \text{K)}$ ， $h_m = 8.0 \times 10^{-7} \text{ m/s}$ 。由于温度方程中不含 C ，水分方程中的 D_1 也不依赖 T ，故问题 1 的温度场和水分场解耦，可在同一数值框架下分别推进。

6.1.2 模型求解

附件 1 只给出了若干时刻的烘房环境数据。首先采用分段线性插值得到连续的边界函数 $T_a(t)$ 和 $C_a(t)$ ，并将环境温度换算为绝对温度。

随后使用有限体积法离散径向空间。将药材半径均分为 160 段，共设置 161 个径向节点：

$$r_i = i\Delta r, \quad \Delta r = \frac{R}{N} = 1.25 \times 10^{-4} \text{ m} = 0.0125 \text{ cm}, \quad i = 0, 1, \dots, N. \quad (33)$$

为了用同一形式表示温度场和水分场，记

$$(\phi, q, \Gamma) = \begin{cases} (T, \rho_1 c_{p1}, k_1), & \text{温度场,} \\ (C, 1, D_1(C)), & \text{水分场.} \end{cases} \quad (34)$$

对每个环形控制体作通量平衡，内部节点的离散方程可写为

$$q_i \omega_i^r \frac{d\phi_i}{dt} = F_{i+1/2} - F_{i-1/2}, \quad F_{i+1/2} = r_{i+1/2} \Gamma_{i+1/2} \frac{\phi_{i+1} - \phi_i}{\Delta r}, \quad (35)$$

其中，控制体的几何权重和界面传递系数分别为

$$\omega_i^r = \frac{r_{i+1/2}^2 - r_{i-1/2}^2}{2}, \quad \Gamma_{i+1/2} = \frac{2\Gamma_i\Gamma_{i+1}}{\Gamma_i + \Gamma_{i+1}}. \quad (36)$$

界面系数采用调和平均 [2]。圆心处取零通量，药材表面的对流边界则写成

$$F_R^T = Rh(T_a - T_N), \quad F_R^C = Rh_m(C_a - C_N). \quad (37)$$

相邻控制体共用同一个界面通量，因此内部通量会成对抵消，药材整体的热量和水分变化均由表面对流边界决定。

完成空间离散后，将各节点的温度和含水率组成状态向量：

$$\mathbf{y} = [T_0, \dots, T_N, C_0, \dots, C_N]^T, \quad \frac{d\mathbf{y}}{dt} = \mathbf{F}(t, \mathbf{y}), \quad \mathbf{y}(0) = \mathbf{y}_0. \quad (38)$$

至此，原偏微分方程组转化为常微分方程组。考虑到温度和水分的变化速度相差较大，采用适合刚性问题的后向差分公式（BDF）推进时间。其计算格式为

$$\sum_{j=0}^k \alpha_j \mathbf{y}_{n+1-j} = \Delta t_n \beta \mathbf{F}(t_{n+1}, \mathbf{y}_{n+1}), \quad 1 \leq k \leq 5, \quad (39)$$

求解器会根据局部误差自动调整步长和阶次 [3]。计算时将相对误差限和绝对误差限分别设为 10^{-6} 和 10^{-8} ，最大内部步长设为 300 s。求得连续时间数值解后，每隔 1 s、每隔 0.1 cm 提取一次结果并写入 **result1.xlsx**。整体求解流程如图3所示。

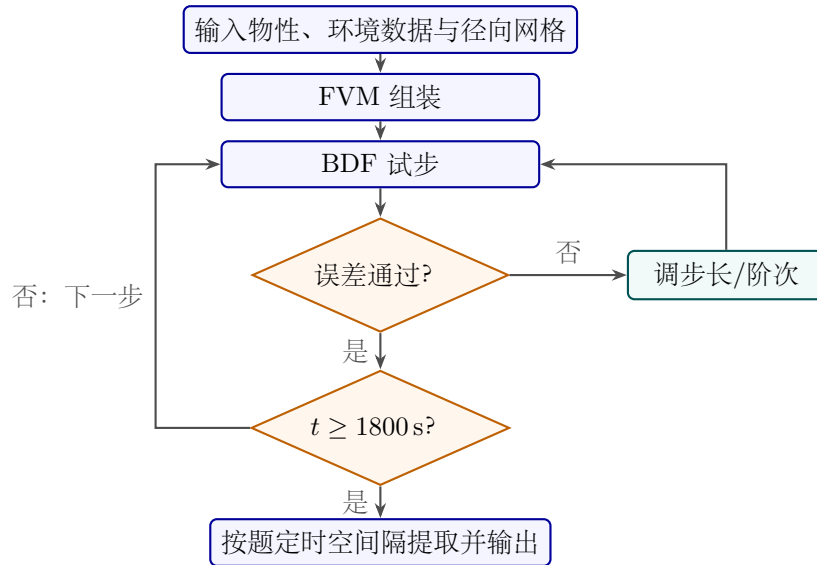


图 3: FVM-BDF 自适应循环求解流程

6.1.3 计算结果与分析

表2与表3分别列出了题目指定时刻和径向位置处的温度与水分浓度，完整的逐秒、逐 0.1 cm 结果保存在 `result1.xlsx` 中。

表 2: 问题 1: 30 min 内药材温度 ($^{\circ}\text{C}$)

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1516	28.3680	28.8476
600	28.4534	28.5361	28.8039	29.3163	30.1662
900	29.3243	29.4583	29.8756	30.6159	31.7306
1200	30.5425	30.7095	31.2220	32.1118	33.4260
1500	31.9954	32.1864	32.7657	33.7462	35.1225
1800	33.5754	33.7721	34.3641	35.3622	36.7859

表 3: 问题 1: 30 min 内水分浓度 (kg/kg)

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2475
300	2.5500	2.5500	2.5500	2.5492	2.0510
600	2.5500	2.5500	2.5500	2.5353	1.8771
900	2.5500	2.5500	2.5497	2.5045	1.7548
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5788
1800	2.5500	2.5497	2.5383	2.3755	1.5103

由表2与表3可知，药材各位置的温度随时间持续升高，并保持表面高、中心低的径向分布；至 1800 s 时，表面温度比中心高 3.2105°C 。水分则首先从外层逸出，表面含水率已较初值下降 1.0397 kg/kg ，而中心含水率在四位小数下仍为初值。图4与图5进一步表明，30 min 内温度扰动已明显向内部传播，水分变化仍主要集中在药材外层，说明该阶段热扩散快于水分扩散。

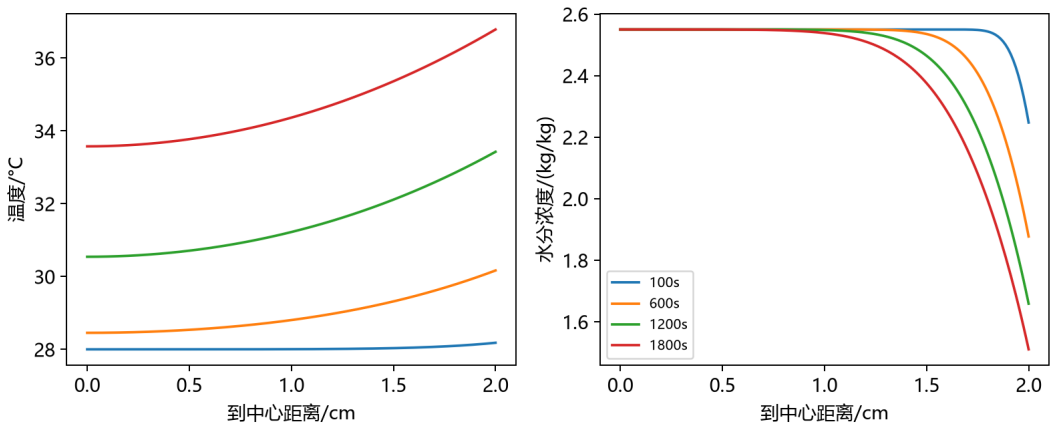


图 4: 问题 1 在不同代表时刻的径向温度与水分浓度分布剖面

问题一完整时空演化

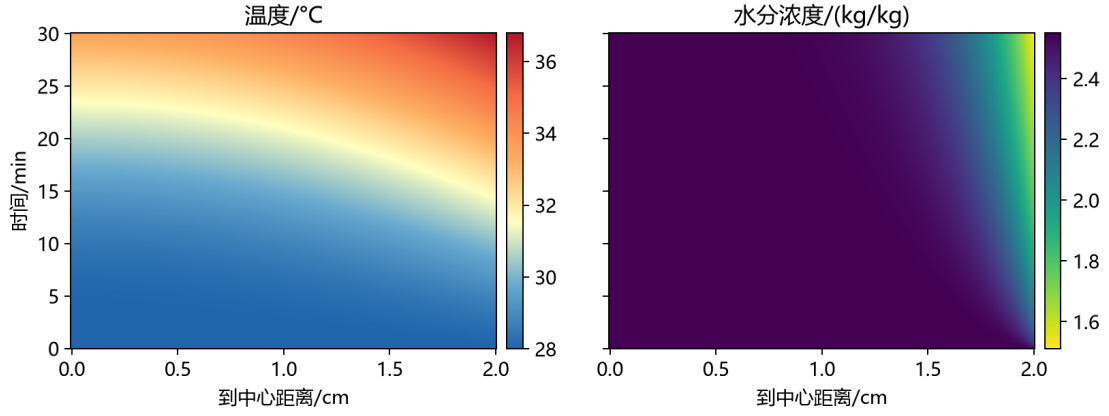


图 5: 问题 1 前 30 min 温度场与水分浓度场的二维时空演化热力图

6.2 问题 2: 非线性物性下的耦合演化

6.2.1 模型求解

在问题 1 中，我们将材料的各项物性参数看作常数，并据此求出了物料内部温度和含水率随时间的变化。问题 2 在此基础上进一步考虑物性参数的变化，即材料的密度、比热容、导热系数和水分扩散系数均会随着温度或含水率发生改变。根据附录 3，问题 2 中各物性参数的计算式为

$$\begin{aligned}\rho_2(C) &= 650 + 128C, \\ c_{p2}(C) &= 1450 + 2736 \frac{C}{C+1}, \\ k_2(C) &= 0.21 + 0.38 \frac{C}{C+1}, \\ D_2(C, T) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T}\right).\end{aligned}\tag{40}$$

可以看出，密度、比热容和导热系数均与含水率有关，而水分扩散系数同时受到温度和含水率的影响。因此，在物料干燥的过程中，温度和含水率发生变化后，各项物性参数也会随之改变，进而影响下一时刻的传热和传质过程。因而，在求解时，不能再将这些参数预先设为固定值，而需要根据每一时刻的温度和含水率重新计算。将式(40)代入式(32)，即可得到问题 2 所需的温度场-水分场联立模型。

在前面的求解过程中，我们已经完成了控制方程和边界条件的处理。因此，对于问题 2，我们只需在每个时间步内增加物性参数的更新过程。具体求解步骤如下：

Step1: 输入问题 1 中已经确定的初始条件和边界条件，并沿物料半径方向设置 201 个节点。中心位置仍采用对称边界条件，物料表面采用相应的传热和传质边界条件。

Step2: 根据当前时刻各节点的温度 T_i 和含水率 C_i ，利用式(40)分别计算该节点处的 $\rho_{2,i}$ 、 $c_{p2,i}$ 、 $k_{2,i}$ 和 $D_{2,i}$ 。

Step3: 将更新后的物性参数代入传热传质方程, 采用与问题 1 相同的有限体积法对空间进行离散, 并用 BDF 方法求出下一时刻各节点的温度和含水率。若该时间步的计算结果满足收敛条件, 则继续计算下一时刻; 否则, 需再次更新物性参数并进行求解。

Step4: 重复上述过程, 直到计算时间达到 10800 s。最后输出物料在代表时刻的径向分布曲线以及前 3 h 内的温度场和水分场变化情况, 并对物料中心和表面的计算结果进行比较。

6.2.2 求解结果

使用上述方法对问题 2 进行计算。当 $t = 10800$ s 时, 物料中心温度为 49.8484 °C, 表面温度为 49.8975 °C, 二者仅相差 0.0491 °C。由此可以看出, 经过 3 h 后, 物料中心和表面的温度已经十分接近, 温度在径向上的差异较小。

在同一时刻, 物料中心处的含水率为 1.7658 kg/kg, 表面处的含水率为 1.0080 kg/kg, 二者相差 0.7578 kg/kg。与温度的计算结果相比, 此时中心和表面的含水率仍有较大差异, 说明物料内部的水分尚未充分向表面迁移。

物料在代表时刻的温度和含水率径向分布如图6所示。从图中可以看出, 温度曲线已经较为平缓, 而含水率从中心到表面呈现出较为明显的下降趋势, 这与上述计算结果相符。

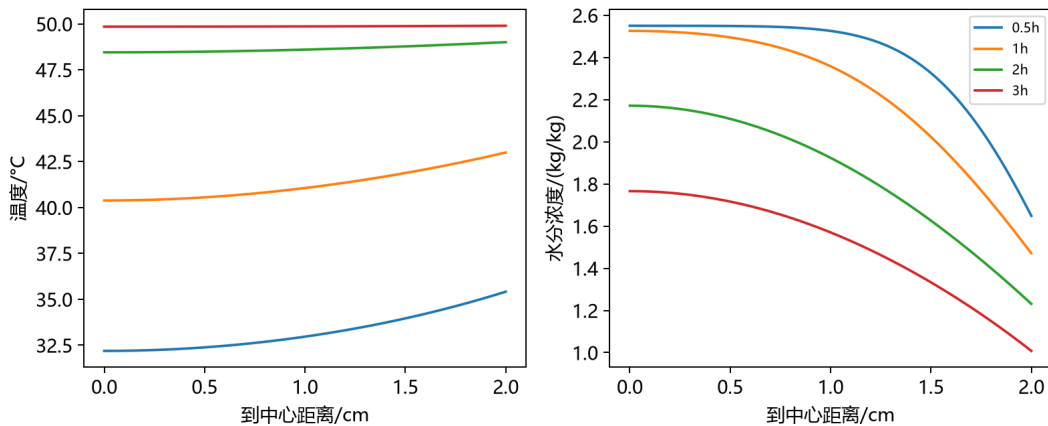


图 6: 问题 2 在代表时刻的径向温度与含水率分布

前 3 h 内温度场和水分场的变化情况如图7所示。从图可以看出, 随着加热时间的增加, 物料各位置温度的差异逐渐减小; 而相比之下, 含水率的变化则要慢得多, 直至计算结束时, 中心处的含水率仍高于表面。这是由于水分扩散系数 $D_2(C, T)$ 会随着温度和含水率的变化而改变, 使得物料内部不同位置的水分迁移速度并不相同。因此, 在前 3 h 内, 物料的温度已基本趋于均匀, 而水分仍需较长时间才能由内部逐渐向外扩散。

问题二完整时空演化

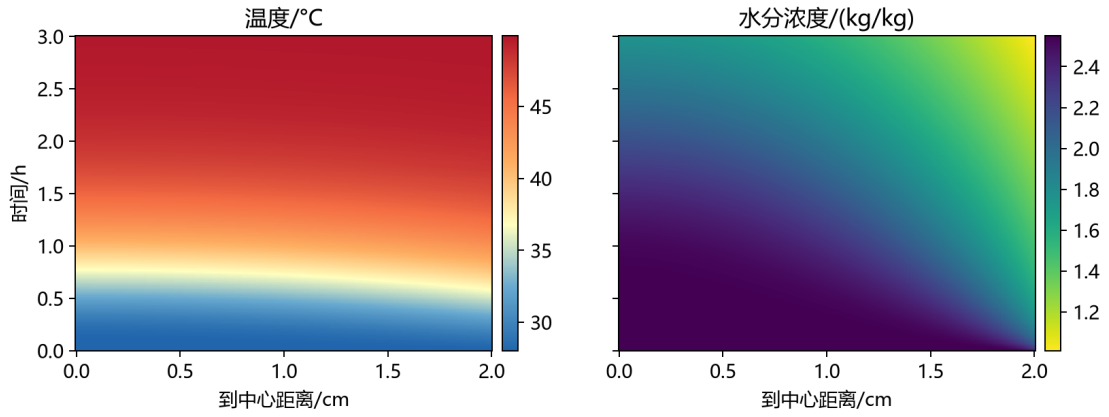


图 7: 问题 2 前 3 h 温度场与水分场的二维时空分布

综上所述，在充分考虑物料的非线性物性之后，它的温度的变化速度仍明显快于水分向外迁移的速度。经过 3 h 计算，物料内部温度已经较为均匀，但中心和表面的含水率仍存在较大差异。因此，在后续判断物料是否达到干燥要求时，不能只考虑温度变化，还需要关注内部水分的扩散过程。

6.3 问题 3：固定半径下的干燥终点

6.3.1 问题分析与终止条件

问题 3 是在物料半径保持不变的条件下，求出其全域水分浓度均降至 0.15 以下的最早时刻。本题不用再建立新的传热传质模型，而是直接沿用问题 2 所得到的温度场-水分场联立系统，并将计算时域延长至干燥过程结束。需要注意的是，表面节点较早达到给定阈值并不能说明物料已经完全干燥；只有径向区域内水分浓度的最大值低于阈值时，才可判定整个物料满足要求。

定义全场最大水分浓度

$$M(t) = \max_{0 \leq r \leq R} C(r, t), \quad (41)$$

并构造事件函数

$$g(t) = M(t) - 0.15. \quad (42)$$

由于干燥阶段的控制量为全域最大值，理论临界时刻可表示为

$$t_c = \inf \{t > 0 \mid M(t) \leq 0.15\}, \quad g(t_c) = 0. \quad (43)$$

(1) 式中， R 为物料半径， $C(r, t)$ 为位置 r 、时刻 t 处的水分浓度。

6.3.2 数值求解方法

在问题 2 的求解过程中，我们已经得到了物料内部温度和水分浓度随时间的变化情况。因此，对于问题 3，只需保持物料半径不变，将原有模型的求解时间适当延长，直到物料各处的水分浓度均低于 0.15 即可。考虑到程序每隔 60 s 输出一次结果，理论上的临界时刻不一定恰好落在输出节点上，所以可以先利用事件函数确定其准确位置，再从输出结果中选取一个满足要求的安全时刻。具体求解步骤如下：

Step1: 输入问题 2 中已经确定的模型参数、初始条件和边界条件，保持物料半径 R 不变，并将径向区域划分为 801 个节点。

Step2: 用问题 2 中的温度场-水分场联立方程进行计算。分别求出各径向节点处每时刻的水分浓度，并从中选出最大值 $M(t)$ 。

Step3: 判断 $M(t)$ 是否降至 0.15。若事件函数 $g(t)$ 由正值变为非正值，则说明全域最大水分浓度在这一时间段内达到规定要求，即可进一步在相邻两个计算时刻之间求解 $g(t) = 0$ ，从而得到理论临界时刻 t_c ；若不满足，则需继续延长时间进行计算。

Step4: 由于题目结果在 60 s 时间网格上给出，还需在 $t_n = 60n$ 处进行再次判断。若某一输出时刻满足 $M(t_n) < 0.15$ ，则将其作为最终的安全达标时刻；否则，需继续考察下一个输出时刻。即

$$t_3 = \min \left\{ 60n \mid \max_{0 \leq j \leq 800} C(r_j, 60n) < 0.15 \right\}. \quad (44)$$

Step5: 输出最终达标的时刻，并记录物料表面和中心位置达到 0.15 的时间。最后结合温度场和水分场的变化图，对干燥终点进行分析。

通过上述步骤，既可以得到水分浓度恰好达到 0.15 的理论时刻，又能保证所取的离散输出时刻已经满足全域水分浓度低于 0.15 的要求。

6.3.3 计算结果与分析

通过数值计算可得，事件函数首次到达零点的连续临界时刻为

$$t_c = 57.5685 \text{ h} = 207246.6 \text{ s}. \quad (45)$$

在 60 s 输出网格上继续进行检验，首个满足全域水分浓度严格低于 0.15 的安全时刻为

$$t_3 = 207300 \text{ s} = 57.5833 \text{ h} \approx 2.40 \text{ d}. \quad (46)$$

此时全域最大水分浓度为 0.149984，比规定阈值低 1.6×10^{-5} 。两级判停结果如表4所示。

表 4: 问题 3 干燥终点计算结果

判定方式	达标时刻/s	达标时刻/h	全域最大水分浓度
连续事件定位	207246.6	57.5685	0.150000
60 s 网格复核	207300	57.5833	0.149984

由表4可以看出，两种方法所得时刻仅相差 53.4 s，小于程序的一个输出步长，因而选取 207300 s 作为最终结果是合理的。此外，通过对物料不同位置的水分浓度进行比较可以发现，物料表面处的水分浓度在 12.55 h 左右已经降至 0.15，而中心处的水分浓度约在 57.58 h 才降至这一数值，前后相差 45.03 h。

这是由于物料表面的水分可以直接向外界扩散，而中心处的水分需要先从物料内部逐渐向表面迁移，这一过程所经过的距离较长，因而下降速度较慢。因此，整个物料何时能够达到干燥要求，这主要取决于中心位置的水分浓度。如果只根据表面位置进行判断，所得干燥时间将会偏小。

全域最大水分浓度随时间的变化曲线如图8所示。由图8可以看出，随着干燥时间的增加， $M(t)$ 不断减小，并在 57.5685 h 附近降至 0.15。此时继续计算至下一个 60 s 输出时刻，得到的全域最大水分浓度为 0.149984，已经满足题目要求。

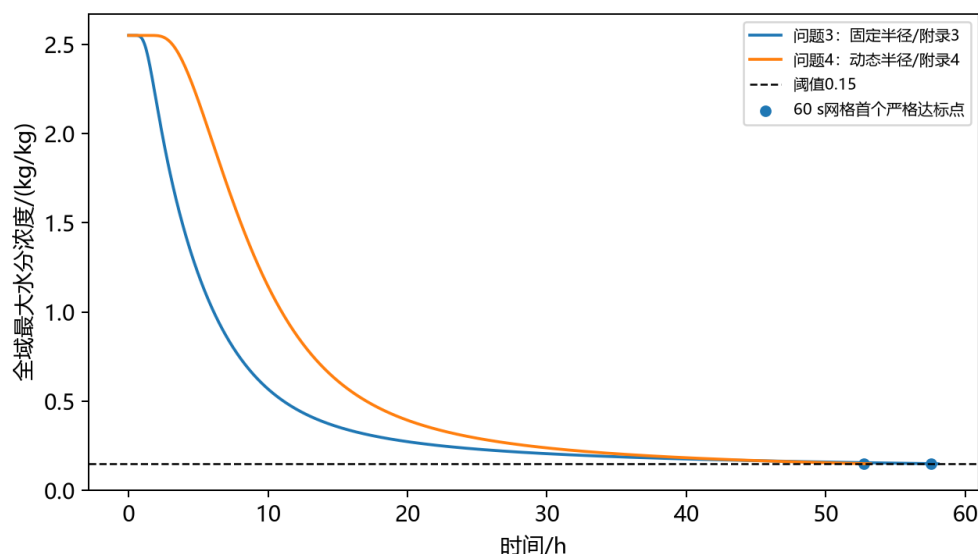


图 8: 全域最大水分浓度随时间的演化及 60 s 安全达标点定位

图9 给出了物料达到干燥要求前后温度场和水分场的变化情况。从图中可以看出，干燥开始后，物料表面处的水分浓度下降较快，而中心处下降相对较慢。在干燥后期，物料内部仍存在一定的水分浓度差异，中心处的水分浓度最后达到规定值，这与前面所得表面和中心位置的达标时间相符。

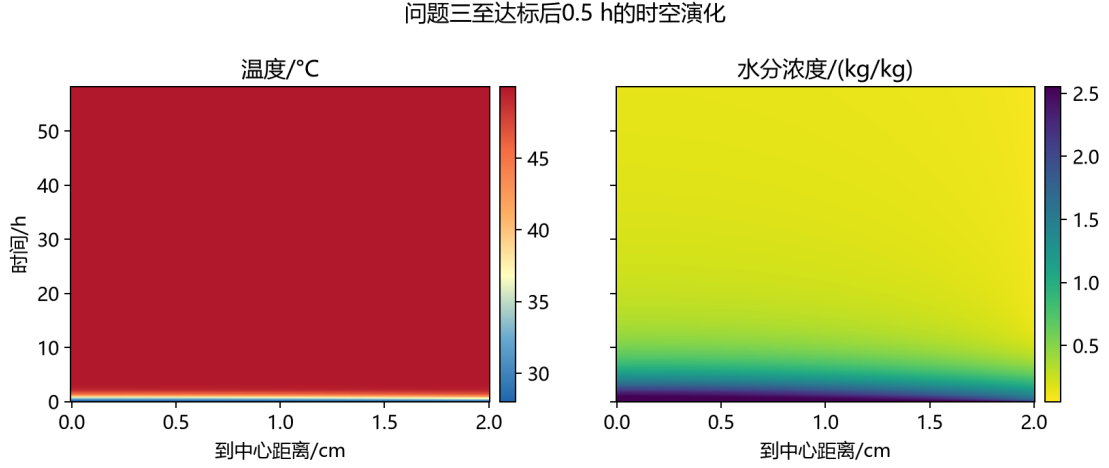


图 9: 问题 3 直至达标后 0.5 h 的温度场与水分场二维时空演化热力图

根据上述计算, 在固定半径条件下, 物料达到干燥要求的时间为 207300 s。由于中心处的水分浓度下降最慢, 因而只有当物料各位置水分浓度的最大值低于 0.15 时, 才说明整个物料已经达到要求。如果只考察物料表面位置的水分浓度, 则会过早结束计算, 所得到的干燥时间也会小于实际所需时间。

6.4 问题 4: 收缩域下的干燥终点

6.4.1 模型建立

问题 4 采用附录 4 的物性函数

$$\begin{aligned}
 \rho_4(C) &= 760 + 90C, \\
 c_{p4}(C) &= 1850 + 2150 \frac{C}{C+1}, \\
 k_4(C) &= 0.12 + 0.20 \frac{C}{C+1}, \\
 D_4(C, T) &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T}\right).
 \end{aligned} \tag{47}$$

同时半径按附件 2 数据随时间收缩。为避免移动网格, 引入相对坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1, \tag{48}$$

并在相邻测量点 (t_j, R_j) 间令

$$R(t) = R_j + \frac{R_{j+1} - R_j}{t_{j+1} - t_j}(t - t_j), \quad \dot{R}(t) = \frac{R_{j+1} - R_j}{t_{j+1} - t_j}. \tag{49}$$

由 $\partial_t \phi|_r = \partial_t \phi|_\xi - \xi(\dot{R}/R)\partial_\xi \phi$ 和 $\partial_r = R^{-1}\partial_\xi$ ，原方程化为固定域上的联立系统

$$\frac{\partial T}{\partial t} = \xi \frac{\dot{R}}{R} \frac{\partial T}{\partial \xi} + \frac{1}{\rho_4 c_{p4} R^2 \xi} \frac{\partial}{\partial \xi} \left(\xi k_4 \frac{\partial T}{\partial \xi} \right), \quad (50)$$

$$\frac{\partial C}{\partial t} = \xi \frac{\dot{R}}{R} \frac{\partial C}{\partial \xi} + \frac{1}{R^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D_4 \frac{\partial C}{\partial \xi} \right). \quad (51)$$

在 $\xi = 0$ 处取对称边界，在 $\xi = 1$ 处联立

$$-\frac{k_4}{R} \frac{\partial T}{\partial \xi} = h(T_s - T_a), \quad -\frac{D_4}{R} \frac{\partial C}{\partial \xi} = h_m(C_s - C_a), \quad (52)$$

并沿用式(42)的终止条件。

6.4.2 模型求解

采用 601 个相对坐标节点，将温度场和水分场在固定区间 $0 \leq \xi \leq 1$ 上作有限体积分散，再使用与前三问相同的 BDF 流程求解。计算过程中根据附件 2 实时更新 $R(t)$ 与 $\dot{R}(t)$ ，并计入收缩引起的几何输运项。

6.4.3 计算结果与分析

连续临界时刻为 52.7135 h；60 s 输出网格上的安全达标时刻及对应半径为

$$\boxed{t_4 = 189780 \text{ s} = 52.7167 \text{ h} \approx 2.20 \text{ d}}, \quad R(t_4) = 1.2000 \text{ cm}. \quad (53)$$

此时全域最大含水率为 0.149994。固定物理采样点若满足 $r_j > R(t)$ ，则已位于收缩后的外部空气域，按题意在表格中留空；“药材表面”列始终取 $\xi = 1$ 处的值。结果见图10，机理对照见第7.3节。

表 5: 问题 4 每 6 h 及达标时刻的水分浓度

时间/h	半径/cm	0 cm	0.5 cm	1.0 cm	药材表面
6	1.3740	1.9539	1.7844	1.2585	0.5678
12	1.2480	0.8748	0.7776	0.4866	0.2147
18	1.2140	0.4600	0.4136	0.2655	0.1021
24	1.2040	0.3076	0.2809	0.1904	0.0713
30	1.2010	0.2383	0.2200	0.1556	0.0609
36	1.2000	0.2002	0.1862	0.1357	0.0566
42	1.2000	0.1762	0.1648	0.1227	0.0544
48	1.2000	0.1597	0.1501	0.1136	0.0531
52.7167	1.2000	0.1500	0.1413	0.1081	0.0525

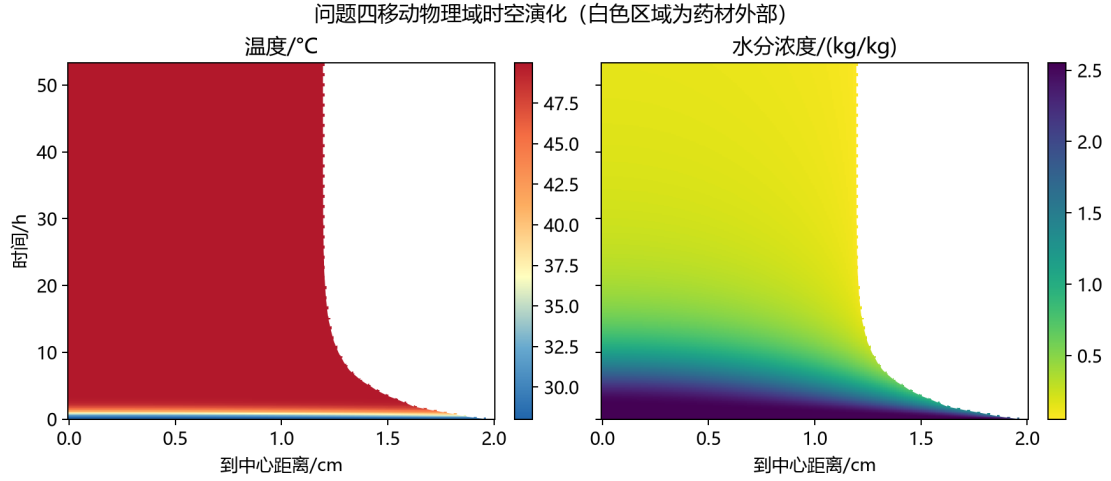


图 10: 问题 4 移动物理域温湿场时空演化热力图（白色区域为药材收缩后的外部空间）

7 模型检验与评价

7.1 数值算法可靠性

随着径向网格的加密，各问计算结果收敛至高精度极限。问题 3 将径向网格由 401 节点逐步加密至 801 节点与 1201 节点，干燥时长由 57.75 h 收敛至 57.57 h 与 57.54 h，加密相对变化仅为 0.043%；问题 4 由 321 节点加密至 601 节点与 801 节点，时长稳定在 52.71 h，相对变化仅为 0.007%。具体网格测试数据见表6，可以充分证明：正式计算所选取的网格密度已处于渐近网格无关区。

表 6: 干燥时长的空间网格收敛数据汇总

问题 3: 划分物理半径 r			问题 4: 划分相对坐标 ξ		
节点数	空间步长 $\Delta r/\text{cm}$	达标时长/h	节点数	初始间距/cm	达标时长/h
401	0.005000	57.7506	321	0.006250	52.7366
801	0.002500	57.5685	601	0.003333	52.7135
1201	0.001667	57.5439	801	0.002500	52.7097

7.2 参数敏感性分析

运行参数敏感性分析模块 (`sensitivity_analysis.py`)，分别对 h 、 h_m 和 D 施加 $\pm 10\%$ 的独立局部摄动，对问题 3、4 达标时间的影响见表7。

表 7: 核心参数的局部敏感性分析结果

改变的参数	问题 3 影响	问题 4 影响	评价
h 减小 10%	小于 0.02%	小于 0.02%	基本不敏感，外部换热不是控速环节
h 增大 10%	小于 0.02%	小于 0.02%	
h_m 减小 10%	+1.26%	+0.59%	弱敏感，表面对流传质仅产生次要影响
h_m 增大 10%	-0.99%	-0.45%	
D 减小 10%	+9.86%	+9.65%	高度敏感，内部水分扩散是主导控速环节
D 增大 10%	-8.03%	-7.88%	

由表7可见， h 的变化几乎不影响干燥终点， h_m 仅产生较弱影响，而 D 改变后两问的达标时间近似同比变化。因此，内部水分向外迁移的扩散阻力是烘干过程的主要控速环节；工艺优化应优先强化内部传质。

7.3 几何收缩机制解耦讨论

在问题 4 中，烘干进程受到“物性改变”与“半径收缩”的双重耦合影响。为辨识各因素的贡献，设立三组对照进行解耦分析（见图11与图12）：

表 8: 几何收缩机制解耦算例

算例	条件	达标时长	评价
A	附录 3 物性，固定半径 2 cm	57.5833 h	作为问题 3 基准结果
B	附录 4 物性，固定半径 2 cm	130.0167 h	相比 A 增加 125.79%，内部扩散阻力显著增大
C	附录 4 物性，半径动态收缩	52.7167 h	相比 B 缩短 77.3 h，降幅 59.45%，整体速率提升 2.47 倍

算例 B 与算例 C 的剧烈反差证明：几何收缩显著缩短了物料内部水分向表面析出的特征扩散距离（ $\tau \propto R^2$ ），是抵消材料本征大扩散阻力、加速干燥完成的最关键机制。若在烘干建模中忽略物料收缩，将会产生高达数天的严重滞后误差。

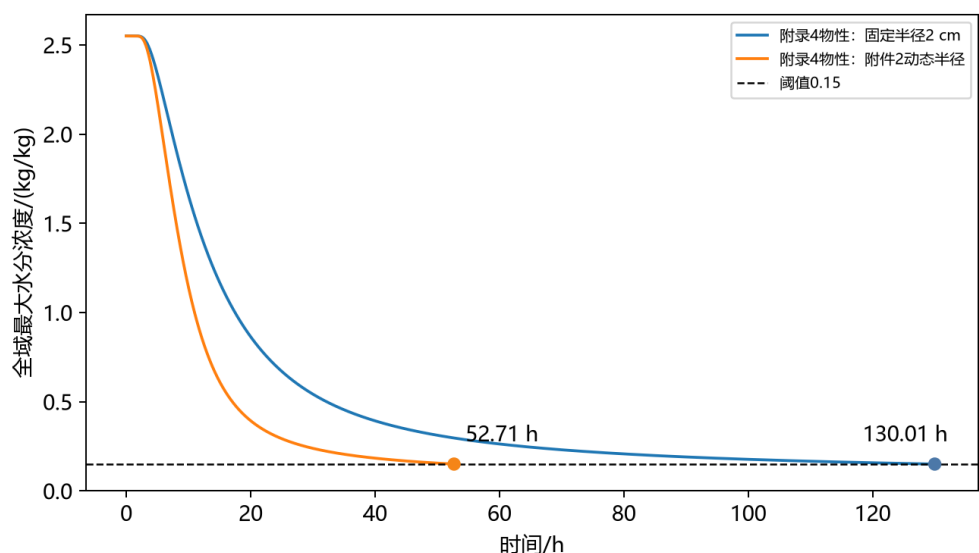


图 11: 附录 4 物性下固定半径与动态收缩的干燥历程对比

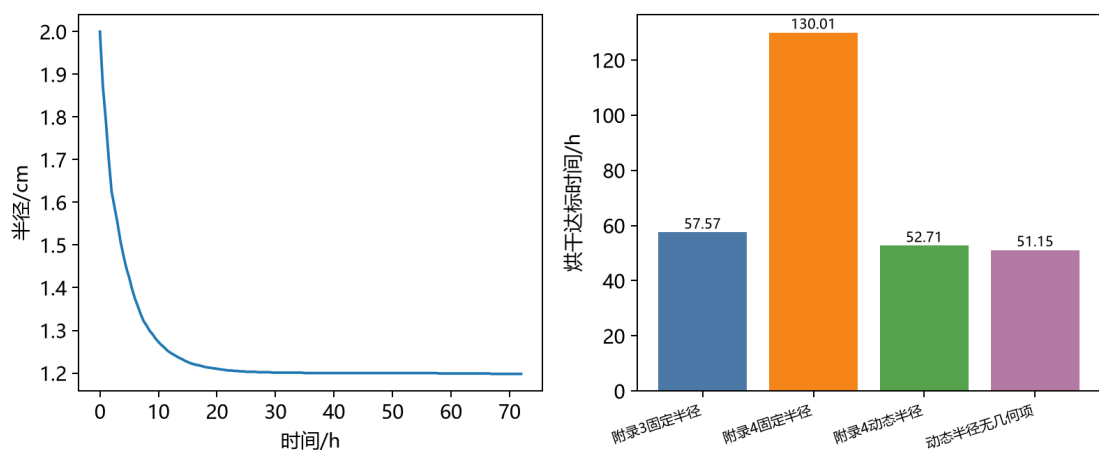


图 12: 附件 2 药材半径实测变化曲线（左）与不同模型烘干达标时间对比柱状图（右）

7.4 模型评价与展望

7.4.1 模型优点

- (1) **框架高度统一可复用**: 四问严格共享同一组热质耦合方程组, 和同一套空间半离散线数值计算方法, 仅需切换对应的物性函数与边界条件, 算法架构整洁优美;
- (2) **自然满足通量条件**: 空间离散采用有限体积法, 内部界面通量形成严格的望远镜级数对消, 机器浮点精度级别满足质量与能量守恒;
- (3) **超刚性方程的高效求解**: 引入两场三点紧凑模式的稀疏雅可比矩阵优化 (CSR), 配合自适应高阶 BDF 隐式积分推进, 在保证绝对数值稳定性的前提下, 实现长达 72 h 跨度物理全过程的秒级极速求解;
- (4) **移动边界处理兼备严格性与实用性**: 通过 Landau 相对坐标归一化彻底规避了动网

格重划分的插值误差，保留了几何收缩双重机制，并构建了连续事件与 60 s 输出网格双重达标判定体系。

7.4.2 局限性与工业优化改进方向

- (1) **相变潜热与热湿焓耦合效应未显式刻画**：模型将水分相变蒸发潜热并入了等效比热容与传热经验参数中，未来可进一步建立包含气-液-固三相平衡的非平衡态升华与汽化能量方程；
- (2) **几何收缩本构关系有待内生**：当前问题 4 基于实测几何数据驱动 $R(t)$ 。若能获得材料力学实验数据，可建立应力-应变与含水率直接关联的收缩本构方程 $R(C)$ ，实现热-质-固力学全耦合求解；
- (3) **二维轴对称与端部效应深化**：对于长径比较小的短柱药材，两端面的吸热与排湿不可忽视，可进一步拓展至 (r, z) 二维有限体积求解器。

7.5 结论

- (1) 烘干初期温度场在 30 min 内迅速建立并在 3 h 内高度接近热平衡，而水分场的迁移具有显著的时间滞后性，全过程由中心最晚达到阈值；
- (2) 问题 3 固定半径下的全域烘干达标时刻为 57.5833 h（约 2.40 d）；问题 4 在附录 4 物性与动态收缩下的达标时刻为 52.7167 h（约 2.20 d），终点尺寸为 1.2000 cm；
- (3) 几何收缩是克服材料内部大扩散阻力的最关键驱动机制，能使干燥整体提速达 2.47 倍。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于【资料整理，代码调试】，详细使用情况见支撑材料。

参考文献

- [1] Crank J. *The Mathematics of Diffusion*. 2nd ed. Oxford: Clarendon Press, 1975.
- [2] Patankar S V. *Numerical Heat Transfer and Fluid Flow*. New York: Hemisphere, 1980.
- [3] Hairer E, Wanner G. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. 2nd ed. Berlin: Springer, 1996.
- [4] Incropera F P, DeWitt D P, Bergman T L, Lavine A S. *Fundamentals of Heat and Mass Transfer*. 6th ed. Hoboken: Wiley, 2007.

[5] Mujumdar A S, ed. *Handbook of Industrial Drying*. 4th ed. Boca Raton: CRC Press, 2014.

支撑材料清单

类别	相对路径 / 文件	内容与功能说明
原始数据	data/附件 1.xlsx	烘房环境温度与水分浓度监测数据
	data/附件 2.xlsx	药材外半径动态变化实测数据
计算程序	run_all.py	全流程一键复现主入口（求解、导出、绘图与校验）
	validate_results.py	结果文件格式规范与物理合理性独立校验程序
	src/	模块化核心求解源码包（物性、FVM、BDF 与采样等）
规定结果	result/result1.xlsx ~ result4.xlsx	问题一至问题四规定格式的数值结果工作簿
	result/summary.json	关键采样点、达标时刻及网格统计数据汇总
	result/parameter_sensitivity.json	传热、传质与扩散系数正负 10% 敏感性分析数据
	result/validation.json	格式规范、边界条件及物理守恒性自动校验报告
说明与图表	report/figures/	论文高清图表（时空演化、径向曲线与达标判据等）
	README.md, requirements.txt	运行环境说明、执行指南及 Python 依赖库清单
	AI 使用情况说明.pdf	本次竞赛 AI 工具使用场景、Prompt 及核验说明
	支撑材料文件清单.pdf	提交文件目录结构、哈希校验摘要与复现指南

复现说明：在支撑材料根目录下安装依赖后，执行 `python run_all.py` 即可一键完成四问数值求解、结果导出、图表绘制、敏感性分析与完整性校验；若仅需复现四问计算结果与图表，可执行 `python run_all.py --skip-sensitivity`。

A 完整源程序代码

本附录收录建模、数值求解、结果导出、绘图、敏感性分析与完整性校验所用的全部 Python 源程序，共计 14 个文件。各文件与支撑材料中的对应文件一致。安装 requirements.txt 中所列依赖后，在支撑材料根目录执行 `python run_all.py`，即可完整复现问题一至问题四的计算结果、Excel 文件、论文图表和校验报告。

A.1 一键运行入口：run_all.py

```
1 """一键复现 A 题四问结果、图表与敏感性分析。"""
2
3 from __future__ import annotations
4
5 import argparse
6
7 from src import sensitivity_analysis, workflow
8 import validate_results
9
10
11 def main() -> None:
12     parser = argparse.ArgumentParser(description=__doc__)
13     parser.add_argument(
14         "--skip-sensitivity",
15         action="store_true",
16         help="跳过参数敏感性分析，仅复现四问结果与图表",
17     )
18     args = parser.parse_args()
19
20     print("=== A 题支撑材料复现开始 ===", flush=True)
21     workflow.run_all(write_excel=True)
22     if not args.skip_sensitivity:
23         print("=== 参数敏感性分析 ===", flush=True)
24         sensitivity_analysis.run_sensitivity(print_output=False)
25     else:
26         print("=== 已按参数跳过敏感性分析 ===", flush=True)
27     print("=== 结果完整性校验 ===", flush=True)
28     validate_results.main(require_sensitivity=not args.skip_sensitivity)
29     print("=== 复现完成 ===", flush=True)
30
31
32 if __name__ == "__main__":
33     main()
```

A.2 项目配置：src/config.py

```
1 """项目路径与全局数值配置。"""
2
3 from pathlib import Path
4
5 PROJECT_ROOT = Path(__file__).resolve().parents[1]
6 DATA_DIR = PROJECT_ROOT / "data"
```

```

7 ATTACHMENT_DIR = DATA_DIR
8 RESULT_DIR = PROJECT_ROOT / "result"
9 TEMPLATE_DIR = RESULT_DIR
10 FIGURE_DIR = PROJECT_ROOT / "report" / "figures"
11
12 INITIAL_T_K = 28.0 + 273.15
13 INITIAL_C = 2.55
14 INITIAL_RADIUS_M = 0.02
15 PLATEAU_START_S = 7200.0
16 RADIUS_MAX_TIME_S = 72.0 * 3600.0
17 FIXED_COMPARISON_MAX_TIME_S = 10.0 * 24.0 * 3600.0
18 POST_DRY_S = 1800.0
19
20 HEAT_TRANSFER_DEFAULT = 25.0
21 MASS_TRANSFER_DEFAULT = 8.0e-7
22 DIFFUSIVITY_SCALE_DEFAULT = 1.0

```

A.3 数据结构: src/models.py

```

1 """求解过程中使用的数据结构。"""
2
3 from dataclasses import dataclass
4
5 import numpy as np
6
7 from .config import PLATEAU_START_S
8
9 @dataclass(frozen=True)
10 class Environment:
11     time_s: np.ndarray
12     temperature_k: np.ndarray
13     moisture: np.ndarray
14     plateau_temperature_k: float
15     plateau_moisture: float
16
17     def __call__(self, t: float) -> tuple[float, float]:
18         if t < PLATEAU_START_S:
19             ta = float(np.interp(t, self.time_s, self.temperature_k))
20             ca = float(np.interp(t, self.time_s, self.moisture))
21             return ta, ca
22         return self.plateau_temperature_k, self.plateau_moisture
23
24
25 @dataclass(frozen=True)
26 class RadiusHistory:
27     time_s: np.ndarray
28     radius_m: np.ndarray
29
30     def value_and_rate(self, t: float) -> tuple[float, float]:
31         if t < self.time_s[0] - 1e-9 or t > self.time_s[-1] + 1e-9:
32             raise ValueError(f"半径查询时间 {t} s 超出附件 2 的 0--72 h 范围")
33         tc = float(np.clip(t, self.time_s[0], self.time_s[-1]))
34         idx = int(np.searchsorted(self.time_s, tc, side="right") - 1)
35         idx = min(max(idx, 0), len(self.time_s) - 2)

```

```

36     t0, t1 = self.time_s[idx], self.time_s[idx + 1]
37     r0, r1 = self.radius_m[idx], self.radius_m[idx + 1]
38     rate = float((r1 - r0) / (t1 - t0))
39     radius = float(r0 + rate * (tc - t0))
40     return radius, rate
41
42
43 @dataclass
44 class Simulation:
45     name: str
46     grid: np.ndarray
47     solution: object
48     moving_radius: bool = False
49
50     @property
51     def n(self) -> int:
52         return self.grid.size
53
54     def fields(self, time_s: np.ndarray | float) -> tuple[np.ndarray, np.ndarray]:
55         values = np.asarray(self.solution.sol(time_s), dtype=float)
56         return values[: self.n], values[self.n :]
```

A.4 附件读取与校验: src/data_io.py

```

1  """读取并校验题目附件数据。"""
2
3  from pathlib import Path
4
5  import numpy as np
6  from openpyxl import load_workbook
7
8  from .config import PLATEAU_START_S, RADIUS_MAX_TIME_S
9  from .models import Environment, RadiusHistory
10
11 def read_environment(path: Path) -> Environment:
12     wb = load_workbook(path, data_only=True, read_only=True)
13     ws = wb.active
14     rows = list(ws.iter_rows(min_row=2, values_only=True))
15     wb.close()
16     time_s = np.asarray([row[0] for row in rows], dtype=float)
17     temperature_k = np.asarray([row[1] for row in rows], dtype=float) + 273.15
18     moisture = np.asarray([row[2] for row in rows], dtype=float)
19     mask = time_s >= PLATEAU_START_S
20     return Environment(
21         time_s=time_s,
22         temperature_k=temperature_k,
23         moisture=moisture,
24         plateau_temperature_k=float(np.mean(temperature_k[mask])),
25         plateau_moisture=float(np.mean(moisture[mask])),
26     )
27
28
29 def read_radius(path: Path) -> RadiusHistory:
30     wb = load_workbook(path, data_only=True, read_only=True)
```

```

31     ws = wb.active
32     rows = list(ws.iter_rows(min_row=2, values_only=True))
33     wb.close()
34     time_s = np.asarray([row[0] for row in rows], dtype=float)
35     radius_m = np.asarray([row[1] for row in rows], dtype=float) / 100.0
36     if time_s[-1] < RADIUS_MAX_TIME_S:
37         raise ValueError("附件 2 未覆盖 72 h，不能按既定方案求解问题四")
38     if np.any(np.diff(radius_m) > 1e-12):
39         raise ValueError("附件 2 半径并非单调不减，请核对数据")
40     return RadiusHistory(time_s=time_s, radius_m=radius_m)

```

A.5 物性参数: src/physics.py

```

1  """附录2--4物性关系与界面系数计算。"""
2
3  import numpy as np
4
5  from .config import (
6      DIFFUSIVITY_SCALE_DEFAULT,
7      HEAT_TRANSFER_DEFAULT,
8      MASS_TRANSFER_DEFAULT,
9  )
10
11  HEAT_TRANSFER = HEAT_TRANSFER_DEFAULT
12  MASS_TRANSFER = MASS_TRANSFER_DEFAULT
13  DIFFUSIVITY_SCALE = DIFFUSIVITY_SCALE_DEFAULT
14
15  def material_properties(
16      model: str, temperature_k: np.ndarray, moisture: np.ndarray
17  ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
18      c = np.maximum(np.asarray(moisture, dtype=float), 1e-10)
19      t = np.asarray(temperature_k, dtype=float)
20      if model == "appendix2":
21          rho = np.full_like(c, 820.0)
22          cp = np.full_like(c, 2600.0)
23          k = np.full_like(c, 0.36)
24          diffusivity = 7.0e-9 * np.exp(-0.89 / c)
25      elif model == "appendix3":
26          rho = 650.0 + 128.0 * c
27          cp = 1450.0 + 2736.0 * c / (c + 1.0)
28          k = 0.21 + 0.38 * c / (c + 1.0)
29          diffusivity = 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / t)
30      elif model == "appendix4":
31          rho = 760.0 + 90.0 * c
32          cp = 1850.0 + 2150.0 * c / (c + 1.0)
33          k = 0.12 + 0.20 * c / (c + 1.0)
34          diffusivity = 4.2e-4 * np.exp(-0.30 / c) * np.exp(-3850.0 / t)
35      else:
36          raise ValueError(f"未知物性模型: {model}")
37      return rho, cp, k, diffusivity * DIFFUSIVITY_SCALE
38
39
40  def harmonic_mean(values: np.ndarray) -> np.ndarray:
41      left, right = values[:-1], values[1:]

```

```
42     return 2.0 * left * right / np.maximum(left + right, 1e-300)
```

A.6 有限体积离散与 BDF 求解: src/solver.py

```
1  """有限体积半离散与 BDF 时间积分。"""
2
3  from typing import Callable
4
5  import numpy as np
6  from scipy.integrate import solve_ivp
7  from scipy.sparse import lil_matrix
8
9  from . import physics
10 from .config import INITIAL_C, INITIAL_RADIUS_M, INITIAL_T_K
11 from .models import Environment, RadiusHistory, Simulation
12
13 def coupled_jacobian_sparsity(node_count: int):
14     """温度/水分两场、三点空间模板对应的稀疏雅可比结构。"""
15     pattern = lil_matrix((2 * node_count, 2 * node_count), dtype=int)
16     for equation_field in range(2):
17         for variable_field in range(2):
18             row_offset = equation_field * node_count
19             col_offset = variable_field * node_count
20             for i in range(node_count):
21                 for j in range(max(0, i - 1), min(node_count, i + 2)):
22                     pattern[row_offset + i, col_offset + j] = 1
23     return pattern.tocsr()
24
25
26 def make_fixed_rhs(
27     radius_m: float,
28     node_count: int,
29     model: str,
30     environment: Environment,
31 ) -> tuple[np.ndarray, Callable[[float, np.ndarray], np.ndarray]]:
32     r = np.linspace(0.0, radius_m, node_count)
33     dr = float(r[1] - r[0])
34     r_half = 0.5 * (r[:-1] + r[1:])
35
36     def rhs(t: float, y: np.ndarray) -> np.ndarray:
37         temperature = y[:node_count]
38         moisture = y[node_count:]
39         rho, cp, k, diffusivity = physics.material_properties(model, temperature, moisture)
40         k_face = physics.harmonic_mean(k)
41         d_face = physics.harmonic_mean(diffusivity)
42         ta, ca = environment(t)
43
44         dt = np.empty(node_count, dtype=float)
45         dc = np.empty(node_count, dtype=float)
46         dt[0] = 4.0 * k_face[0] * (temperature[1] - temperature[0]) / (
47             rho[0] * cp[0] * dr**2
48         )
49         dc[0] = 4.0 * d_face[0] * (moisture[1] - moisture[0]) / dr**2
50
```

```

51     heat_out = r_half[1:] * k_face[1:] * np.diff(temperature)[1:] / dr
52     heat_in = r_half[:-1] * k_face[:-1] * np.diff(temperature)[:-1] / dr
53     mass_out = r_half[1:] * d_face[1:] * np.diff(moisture)[1:] / dr
54     mass_in = r_half[:-1] * d_face[:-1] * np.diff(moisture)[:-1] / dr
55     dt[1:-1] = (heat_out - heat_in) / (rho[1:-1] * cp[1:-1] * r[1:-1] * dr)
56     dc[1:-1] = (mass_out - mass_in) / (r[1:-1] * dr)
57
58     r_minus = radius_m - 0.5 * dr
59     shell_measure = radius_m**2 - r_minus**2
60     dt[-1] = 2.0 * (
61         -radius_m * physics.HEAT_TRANSFER * (temperature[-1] - ta)
62         - r_minus * k_face[-1] * (temperature[-1] - temperature[-2]) / dr
63     ) / (rho[-1] * cp[-1] * shell_measure)
64     dc[-1] = 2.0 * (
65         -radius_m * physics.MASS_TRANSFER * (moisture[-1] - ca)
66         - r_minus * d_face[-1] * (moisture[-1] - moisture[-2]) / dr
67     ) / shell_measure
68     return np.concatenate((dt, dc))
69
70     return r, rhs
71
72
73 def make_moving_rhs(
74     node_count: int,
75     environment: Environment,
76     radius_history: RadiusHistory,
77     include_geometry_term: bool,
78 ) -> tuple[np.ndarray, Callable[[float, np.ndarray], np.ndarray]]:
79     xi = np.linspace(0.0, 1.0, node_count)
80     dxi = float(xi[1] - xi[0])
81     xi_half = 0.5 * (xi[:-1] + xi[1:])
82
83     def rhs(t: float, y: np.ndarray) -> np.ndarray:
84         temperature = y[:node_count]
85         moisture = y[node_count:]
86         radius, radius_rate = radius_history.value_and_rate(t)
87         rho, cp, k, diffusivity = physics.material_properties(
88             "appendix4", temperature, moisture
89         )
90         k_face = physics.harmonic_mean(k)
91         d_face = physics.harmonic_mean(diffusivity)
92         ta, ca = environment(t)
93
94         dt = np.empty(node_count, dtype=float)
95         dc = np.empty(node_count, dtype=float)
96         dt[0] = 4.0 * k_face[0] * (temperature[1] - temperature[0]) / (
97             rho[0] * cp[0] * radius**2 * dxi**2
98         )
99         dc[0] = 4.0 * d_face[0] * (moisture[1] - moisture[0]) / (
100             radius**2 * dxi**2
101         )
102
103         heat_out = xi_half[1:] * k_face[1:] * np.diff(temperature)[1:] / dxi
104         heat_in = xi_half[:-1] * k_face[:-1] * np.diff(temperature)[:-1] / dxi
105         mass_out = xi_half[1:] * d_face[1:] * np.diff(moisture)[1:] / dxi
106         mass_in = xi_half[:-1] * d_face[:-1] * np.diff(moisture)[:-1] / dxi

```

```

107     dt[1:-1] = (heat_out - heat_in) / (
108         rho[1:-1] * cp[1:-1] * radius**2 * xi[1:-1] * dxi
109     )
110     dc[1:-1] = (mass_out - mass_in) / (
111         radius**2 * xi[1:-1] * dxi
112     )
113
114     xi_minus = 1.0 - 0.5 * dxi
115     shell_measure = 1.0 - xi_minus**2
116     dt[-1] = 2.0 * (
117         -radius * physics.HEAT_TRANSFER * (temperature[-1] - ta)
118         - xi_minus * k_face[-1] * (temperature[-1] - temperature[-2]) / dxi
119     ) / (rho[-1] * cp[-1] * radius**2 * shell_measure)
120     dc[-1] = 2.0 * (
121         -radius * physics.MASS_TRANSFER * (moisture[-1] - ca)
122         - xi_minus * d_face[-1] * (moisture[-1] - moisture[-2]) / dxi
123     ) / (radius**2 * shell_measure)
124
125     if include_geometry_term:
126         factor = radius_rate / radius
127         t_xi = np.empty(node_count, dtype=float)
128         c_xi = np.empty(node_count, dtype=float)
129         t_xi[0] = 0.0
130         c_xi[0] = 0.0
131         t_xi[1:-1] = (temperature[2:] - temperature[:-2]) / (2.0 * dxi)
132         c_xi[1:-1] = (moisture[2:] - moisture[:-2]) / (2.0 * dxi)
133         t_xi[-1] = -radius * physics.HEAT_TRANSFER * (temperature[-1] - ta) / k[-1]
134         c_xi[-1] = -radius * physics.MASS_TRANSFER * (moisture[-1] - ca) / diffusivity[-1]
135         dt += xi * factor * t_xi
136         dc += xi * factor * c_xi
137     return np.concatenate((dt, dc))
138
139     return xi, rhs
140
141
142 def dry_event(node_count: int) -> Callable[[float, np.ndarray], float]:
143     def event(_t: float, y: np.ndarray) -> float:
144         return float(np.max(y[node_count:]) - 0.15)
145
146     event.direction = -1
147     event.terminal = False
148     return event
149
150
151 def solve_fixed(
152     name: str,
153     end_s: float,
154     node_count: int,
155     model: str,
156     environment: Environment,
157     track_dry_event: bool = False,
158 ) -> Simulation:
159     grid, rhs = make_fixed_rhs(INITIAL_RADIUS_M, node_count, model, environment)
160     y0 = np.concatenate(
161         (np.full(node_count, INITIAL_T_K), np.full(node_count, INITIAL_C))
162     )

```

```

163     events = dry_event(node_count) if track_dry_event else None
164     solution = solve_ivp(
165         rhs,
166         (0.0, end_s),
167         y0,
168         method="BDF",
169         rtol=1.0e-6,
170         atol=1.0e-8,
171         max_step=300.0,
172         dense_output=True,
173         events=events,
174         jac_sparsity=coupled_jacobian_sparsity(node_count),
175     )
176     if not solution.success:
177         raise RuntimeError(f"{name} 求解失败: {solution.message}")
178     return Simulation(name=name, grid=grid, solution=solution)
179
180
181 def solve_moving(
182     name: str,
183     end_s: float,
184     node_count: int,
185     environment: Environment,
186     radius_history: RadiusHistory,
187     include_geometry_term: bool,
188     track_dry_event: bool = True,
189 ) -> Simulation:
190     grid, rhs = make_moving_rhs(
191         node_count, environment, radius_history, include_geometry_term
192     )
193     y0 = np.concatenate(
194         (np.full(node_count, INITIAL_T_K), np.full(node_count, INITIAL_C))
195     )
196     events = dry_event(node_count) if track_dry_event else None
197     solution = solve_ivp(
198         rhs,
199         (0.0, end_s),
200         y0,
201         method="BDF",
202         rtol=1.0e-6,
203         atol=1.0e-8,
204         max_step=300.0,
205         dense_output=True,
206         events=events,
207         jac_sparsity=coupled_jacobian_sparsity(node_count),
208     )
209     if not solution.success:
210         raise RuntimeError(f"{name} 求解失败: {solution.message}")
211     return Simulation(name=name, grid=grid, solution=solution, moving_radius=True)

```

A.7 阈值判定与空间采样: src/sampling.py

```

1 """数值解的阈值判定、空间采样与网格误差计算。"""
2

```

```

3 import numpy as np
4 from scipy.optimize import brentq
5
6 from .models import RadiusHistory, Simulation
7
8 def event_time(simulation: Simulation) -> float:
9     event_arrays = simulation.solution.t_events
10    if not event_arrays or len(event_arrays[0]) == 0:
11        sample_t = np.linspace(0.0, simulation.solution.t[-1], 2001)
12        _, sample_c = simulation.fields(sample_t)
13        residual = np.max(sample_c, axis=0) - 0.15
14        crossed = np.where((residual[:-1] >= 0.0) & (residual[1:] < 0.0))[0]
15        if crossed.size == 0:
16            end_h = simulation.solution.t[-1] / 3600.0
17            raise RuntimeError(f"{simulation.name} 在 {end_h:g} h 内未达到全域 C<0.15")
18        idx = int(crossed[0])
19
20        def scalar_residual(t: float) -> float:
21            return float(np.max(simulation.fields(t)[1]) - 0.15)
22
23        return float(brentq(scalar_residual, sample_t[idx], sample_t[idx + 1]))
24    return float(event_arrays[0][0])
25
26
27 def sample_fixed(
28     simulation: Simulation, times_s: np.ndarray, output_r_m: np.ndarray
29 ) -> tuple[np.ndarray, np.ndarray]:
30     temperature, moisture = simulation.fields(times_s)
31     sampled_t = np.vstack(
32         [np.interp(output_r_m, simulation.grid, temperature[:, j]) for j in range(times_s.size)]
33     )
34     sampled_c = np.vstack(
35         [np.interp(output_r_m, simulation.grid, moisture[:, j]) for j in range(times_s.size)]
36     )
37     return sampled_t - 273.15, sampled_c
38
39
40 def sample_moving_moisture(
41     simulation: Simulation,
42     radius_history: RadiusHistory,
43     times_s: np.ndarray,
44     output_r_m: np.ndarray,
45 ) -> tuple[np.ndarray, np.ndarray]:
46     _, moisture = simulation.fields(times_s)
47     sampled = np.full((times_s.size, output_r_m.size), np.nan, dtype=float)
48     radii = np.empty(times_s.size, dtype=float)
49     for j, time_s in enumerate(times_s):
50         radius, _ = radius_history.value_and_rate(float(time_s))
51         radii[j] = radius
52         valid = output_r_m <= radius + 1e-12
53         sampled[j, valid] = np.interp(
54             output_r_m[valid] / radius, simulation.grid, moisture[:, j]
55         )
56     return sampled, radii
57
58 def representative_table(

```

```

59     simulation: Simulation, times_h: list[float], distances_cm: list[float]
60 ) -> dict[str, list]:
61     times_s = np.asarray(times_h, dtype=float) * 3600.0
62     radii_m = np.asarray(distances_cm, dtype=float) / 100.0
63     temperature_c, moisture = sample_fixed(simulation, times_s, radii_m)
64     return {
65         "times_h": times_h,
66         "distances_cm": distances_cm,
67         "temperature_c": np.round(temperature_c, 4).tolist(),
68         "moisture": np.round(moisture, 4).tolist(),
69     }
70
71
72 def max_moisture(simulation: Simulation, time_s: float) -> float:
73     return float(np.max(simulation.fields(time_s)[1]))
74
75
76 def first_output_time_below(
77     simulation: Simulation, step_s: float = 60.0, require_displayed_below: bool = False
78 ) -> tuple[float, float]:
79     """返回输出网格上首次严格达标的时刻及未经舍入的最大含水率。"""
80     times = np.arange(step_s, simulation.solution.t[-1] + 0.1, step_s)
81     maxima = np.max(simulation.fields(times)[1], axis=0)
82     eligible = np.round(maxima, 4) < 0.15 if require_displayed_below else maxima < 0.15
83     indices = np.flatnonzero(eligible)
84     if indices.size == 0:
85         qualifier = "四位小数显示" if require_displayed_below else "原始数值"
86         raise RuntimeError(f"{simulation.name} 没有在输出网格上以{qualifier}达到 C<0.15")
87     index = int(indices[0])
88     return float(times[index]), float(maxima[index])
89
90
91 def first_node_output_time_below(
92     simulation: Simulation, node_index: int, step_s: float = 60.0
93 ) -> tuple[float, float]:
94     """返回指定计算节点在输出网格上首次严格低于0.15的时刻。"""
95     times = np.arange(step_s, simulation.solution.t[-1] + 0.1, step_s)
96     values = simulation.fields(times)[1][node_index]
97     indices = np.flatnonzero(values < 0.15)
98     if indices.size == 0:
99         raise RuntimeError(f"{simulation.name} 的节点{node_index}在输出网格上未达标")
100     index = int(indices[0])
101     return float(times[index]), float(values[index])
102
103
104 def grid_difference(
105     coarse: Simulation, fine: Simulation, time_s: float, field: str = "moisture"
106 ) -> float:
107     coarse_t, coarse_c = coarse.fields(time_s)
108     fine_t, fine_c = fine.fields(time_s)
109     coarse_values = coarse_c if field == "moisture" else coarse_t
110     fine_values = fine_c if field == "moisture" else fine_t
111     interpolated = np.interp(coarse.grid, fine.grid, fine_values)
112     return float(np.max(np.abs(coarse_values - interpolated)))

```

A.8 Excel 结果输出: src/excel_output.py

```
1 """按附件3模板格式写出四个结果工作簿。"""
2
3 from pathlib import Path
4
5 import numpy as np
6 from openpyxl import Workbook, load_workbook
7 from openpyxl.styles import Alignment, Font, PatternFill
8
9 def reset_sheet(ws) -> None:
10     if ws.max_row:
11         ws.delete_rows(1, ws.max_row)
12
13
14 def style_header(ws, max_col: int) -> None:
15     fill = PatternFill("solid", fgColor="D9EAF7")
16     for cell in ws[1][:max_col]:
17         cell.font = Font(bold=True)
18         cell.fill = fill
19         cell.alignment = Alignment(horizontal="center")
20     ws.freeze_panes = "B2"
21     ws.column_dimensions["A"].width = 13
22
23
24 def write_two_sheet_result(
25     path: Path,
26     times_s: np.ndarray,
27     distances_cm: np.ndarray,
28     temperature_c: np.ndarray,
29     moisture: np.ndarray,
30 ) -> None:
31     if path.exists():
32         wb = load_workbook(path)
33     else:
34         wb = Workbook()
35         wb.active.title = "温度"
36         wb.create_sheet("水分浓度")
37     mapping = {"温度": temperature_c, "水分浓度": moisture}
38     for sheet_name, values in mapping.items():
39         ws = wb[sheet_name]
40         reset_sheet(ws)
41         ws.cell(1, 1, "时间\\到药材中心的距离")
42         for col, distance in enumerate(distances_cm, start=2):
43             ws.cell(1, col, float(round(distance, 1)))
44         for row, time_s in enumerate(times_s, start=2):
45             ws.cell(row, 1, int(round(float(time_s))))
46             for col, value in enumerate(values[row - 2], start=2):
47                 cell = ws.cell(row, col, float(round(value, 4)))
48                 cell.number_format = "0.0000"
49             style_header(ws, distances_cm.size + 1)
50             ws.auto_filter.ref = ws.dimensions
51         wb.save(path)
52
53
54 def write_single_sheet_result(
```

```

55     path: Path,
56     times_s: np.ndarray,
57     headers: list[float | str],
58     values: np.ndarray,
59 ) -> None:
60     if path.exists():
61         wb = load_workbook(path)
62     else:
63         wb = Workbook()
64         wb.active.title = "Sheet1"
65     ws = wb.active
66     reset_sheet(ws)
67     ws.cell(1, 1, "时间\\到药材中心的距离")
68     for col, header in enumerate(headers, start=2):
69         ws.cell(1, col, header)
70     for row, time_s in enumerate(times_s, start=2):
71         ws.cell(row, 1, int(round(float(time_s))))
72         for col, value in enumerate(values[row - 2], start=2):
73             if np.isfinite(value):
74                 cell = ws.cell(row, col, float(round(value, 4)))
75                 cell.number_format = "0.0000"
76     style_header(ws, len(headers) + 1)
77     ws.auto_filter.ref = ws.dimensions
78     wb.save(path)

```

A.9 论文图表生成: src/plotting.py

```

1  """生成论文所需的数值结果图。"""
2
3  import matplotlib
4
5  matplotlib.use("Agg")
6  import matplotlib.pyplot as plt
7  import numpy as np
8  from matplotlib.colors import LinearSegmentedColormap
9
10 from .config import FIGURE_DIR, INITIAL_RADIUS_M, POST_DRY_S, RADIUS_MAX_TIME_S
11 from .models import Environment, RadiusHistory, Simulation
12 from .sampling import event_time, max_moisture, sample_fixed
13
14 TEMPERATURE_CMAP = LinearSegmentedColormap.from_list(
15     "temperature_blue_yellow_red",
16     ["#2166ac", "#67a9cf", "#ffffbf", "#fdae61", "#f46d43", "#b2182b"],
17 )
18 MOISTURE_CMAP = plt.get_cmap("viridis_r")
19
20 def create_figures(
21     environment: Environment,
22     radius_history: RadiusHistory,
23     q1: Simulation,
24     q2: Simulation,
25     q3: Simulation,
26     q4: Simulation,
27     appendix4_fixed: Simulation,

```

```

28     dry3_s: float,
29     dry4_s: float,
30     safe3_s: float,
31     safe4_s: float,
32     comparison_times_h: dict[str, float],
33 ) -> None:
34     FIGURE_DIR.mkdir(parents=True, exist_ok=True)
35     plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "SimHei"], "axes.unicode_minus": False
36     })
37
38     full_time = np.linspace(0.0, RADIUS_MAX_TIME_S, 721)
39     full_env = np.asarray([environment(float(t)) for t in full_time])
40     fig, axes = plt.subplots(2, 1, figsize=(7.2, 6.0), sharex=True)
41     axes[0].plot(environment.time_s / 3600.0, environment.temperature_k - 273.15, ".", ms=2, label="附件1")
42     axes[0].plot(full_time / 3600.0, full_env[:, 0] - 273.15, lw=1.3, label="插值/平台延拓")
43     axes[0].set_ylabel("环境温度/°C")
44     axes[0].legend()
45     axes[1].plot(environment.time_s / 3600.0, environment.moisture, ".", ms=2)
46     axes[1].plot(full_time / 3600.0, full_env[:, 1], lw=1.3)
47     axes[1].set_xlabel("时间/h")
48     axes[1].set_ylabel("环境水分浓度/(kg/kg)")
49     fig.tight_layout()
50     fig.savefig(FIGURE_DIR / "environment_extension.png", dpi=180)
51     plt.close(fig)
52
53     fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8))
54     for t_s in [100, 600, 1200, 1800]:
55         tc, cc = sample_fixed(q1, np.asarray([t_s]), q1.grid)
56         axes[0].plot(q1.grid * 100, tc[0], label=f"{t_s}s")
57         axes[1].plot(q1.grid * 100, cc[0], label=f"{t_s}s")
58     axes[0].set_xlabel="到中心距离/cm", ylabel="温度/°C"
59     axes[1].set_xlabel="到中心距离/cm", ylabel="水分浓度/(kg/kg)"
60     axes[1].legend(fontsize=8)
61     fig.tight_layout()
62     fig.savefig(FIGURE_DIR / "question1_profiles.png", dpi=180)
63     plt.close(fig)
64
65     fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8))
66     for t_h in [0.5, 1.0, 2.0, 3.0]:
67         tc, cc = sample_fixed(q2, np.asarray([t_h * 3600]), q2.grid)
68         axes[0].plot(q2.grid * 100, tc[0], label=f"{t_h:g}h")
69         axes[1].plot(q2.grid * 100, cc[0], label=f"{t_h:g}h")
70     axes[0].set_xlabel="到中心距离/cm", ylabel="温度/°C"
71     axes[1].set_xlabel="到中心距离/cm", ylabel="水分浓度/(kg/kg)"
72     axes[1].legend(fontsize=8)
73     fig.tight_layout()
74     fig.savefig(FIGURE_DIR / "question2_profiles.png", dpi=180)
75     plt.close(fig)
76
77     def fixed_heatmaps(simulation: Simulation, end_s: float, filename: str, title: str) -> None:
78         plot_times = np.linspace(0.0, end_s, 401)
79         plot_r_m = np.linspace(0.0, INITIAL_RADIUS_M, 241)
80         temperature_c, moisture = sample_fixed(simulation, plot_times, plot_r_m)
81         time_scale = 3600.0 if end_s > 7200.0 else 60.0
82         time_label = "时间/h" if time_scale == 3600.0 else "时间/min"

```

```

82     fig, axes = plt.subplots(1, 2, figsize=(9.4, 4.0), sharey=True)
83     image_t = axes[0].pcolormesh(
84         plot_r_m * 100.0, plot_times / time_scale, temperature_c,
85         shading="auto", cmap=TEMPERATURE_CMAP
86     )
87     image_c = axes[1].pcolormesh(
88         plot_r_m * 100.0, plot_times / time_scale, moisture,
89         shading="auto", cmap=MOISTURE_CMAP
90     )
91     axes[0].set(xlabel="到中心距离/cm", ylabel=time_label, title="温度/°C")
92     axes[1].set(xlabel="到中心距离/cm", title="水分浓度/(kg/kg)")
93     fig.colorbar(image_t, ax=axes[0], pad=0.02)
94     fig.colorbar(image_c, ax=axes[1], pad=0.02)
95     fig.suptitle(title)
96     fig.tight_layout()
97     fig.savefig(FIGURE_DIR / filename, dpi=180)
98     plt.close(fig)
99
100     fixed_heatmaps(q1, 1800.0, "question1_heatmaps.png", "问题一完整时空演化")
101     fixed_heatmaps(q2, 10800.0, "question2_heatmaps.png", "问题二完整时空演化")
102     fixed_heatmaps(q3, safe3_s + POST_DRY_S, "question3_heatmaps.png", "问题三至达标后0.5 h的时空演化")
103
104     q4_plot_times = np.linspace(0.0, safe4_s + POST_DRY_S, 401)
105     q4_plot_r_m = np.linspace(0.0, INITIAL_RADIUS_M, 241)
106     q4_temperature, q4_moisture = q4.fields(q4_plot_times)
107     q4_t_map = np.full((q4_plot_times.size, q4_plot_r_m.size), np.nan)
108     q4_c_map = np.full_like(q4_t_map, np.nan)
109     for j, time_s in enumerate(q4_plot_times):
110         radius, _ = radius_history.value_and_rate(float(time_s))
111         valid = q4_plot_r_m <= radius + 1e-12
112         xi_query = q4_plot_r_m[valid] / radius
113         q4_t_map[j, valid] = np.interp(xi_query, q4.grid, q4_temperature[:, j]) - 273.15
114         q4_c_map[j, valid] = np.interp(xi_query, q4.grid, q4_moisture[:, j])
115     cmap_t = TEMPERATURE_CMAP.copy()
116     cmap_c = MOISTURE_CMAP.copy()
117     cmap_t.set_bad("white")
118     cmap_c.set_bad("white")
119     fig, axes = plt.subplots(1, 2, figsize=(9.6, 4.2), sharey=True, constrained_layout=True)
120     image_t = axes[0].pcolormesh(
121         q4_plot_r_m * 100.0, q4_plot_times / 3600.0, q4_t_map,
122         shading="auto", cmap=cmap_t
123     )
124     image_c = axes[1].pcolormesh(
125         q4_plot_r_m * 100.0, q4_plot_times / 3600.0, q4_c_map,
126         shading="auto", cmap=cmap_c
127     )
128     radius_curve = np.asarray([radius_history.value_and_rate(float(t))[0] for t in q4_plot_times])
129     for axis in axes:
130         axis.plot(radius_curve * 100.0, q4_plot_times / 3600.0, color="white", lw=1.2, ls="--")
131         axis.set_xlabel("到中心距离/cm")
132     axes[0].set(ylabel="时间/h", title="温度/°C")
133     axes[1].set_title("水分浓度/(kg/kg)")
134     fig.colorbar(image_t, ax=axes[0], pad=0.02)
135     fig.colorbar(image_c, ax=axes[1], pad=0.02)
136     fig.suptitle("问题四移动物理域时空演化（白色区域为药材外部）")
137     fig.savefig(FIGURE_DIR / "question4_heatmaps.png", dpi=180, bbox_inches="tight")

```

```

138 plt.close(fig)
139
140 check_time3 = np.linspace(0.0, min(q3.solution.t[-1], dry3_s + POST_DRY_S), 800)
141 check_time4 = np.linspace(0.0, min(q4.solution.t[-1], dry4_s + POST_DRY_S), 500)
142 max3 = np.max(q3.fields(check_time3)[1], axis=0)
143 max4 = np.max(q4.fields(check_time4)[1], axis=0)
144 fig, ax = plt.subplots(figsize=(7.2, 4.2))
145 ax.plot(check_time3 / 3600.0, max3, label="问题3：固定半径/附录3")
146 ax.plot(check_time4 / 3600.0, max4, label="问题4：动态半径/附录4")
147 ax.axhline(0.15, color="black", ls="--", lw=1, label="阈值0.15")
148 safe_values = [max_moisture(q3, safe3_s), max_moisture(q4, safe4_s)]
149 ax.scatter([safe3_s / 3600, safe4_s / 3600], safe_values, s=24, label="60 s网格首个严格达标点")
150 ax.set_xlabel("时间/h", ylabel="全域最大水分浓度/(kg/kg)")
151 ax.set_ylim(bottom=0)
152 ax.legend(fontsize=8)
153 fig.tight_layout()
154 fig.savefig(FIGURE_DIR / "drying_threshold.png", dpi=180)
155 plt.close(fig)
156
157 fixed_end_s = event_time(appendix4_fixed)
158 fixed_times = np.linspace(0.0, min(appendix4_fixed.solution.t[-1], fixed_end_s + POST_DRY_S), 900)
159 moving_times = np.linspace(0.0, min(q4.solution.t[-1], safe4_s + POST_DRY_S), 500)
160 fixed_max = np.max(appendix4_fixed.fields(fixed_times)[1], axis=0)
161 moving_max = np.max(q4.fields(moving_times)[1], axis=0)
162 fig, ax = plt.subplots(figsize=(7.4, 4.3))
163 ax.plot(fixed_times / 3600.0, fixed_max, lw=1.5, label="附录4物性：固定半径2 cm")
164 ax.plot(moving_times / 3600.0, moving_max, lw=1.5, label="附录4物性：附件2动态半径")
165 ax.axhline(0.15, color="black", ls="--", lw=1, label="阈值0.15")
166 ax.scatter(
167     [fixed_end_s / 3600.0, dry4_s / 3600.0], [0.15, 0.15],
168     color=["#4C78A8", "#F58518"], zorder=3
169 )
170 ax.annotate(f"{fixed_end_s / 3600.0:.2f} h", (fixed_end_s / 3600.0, 0.15), xytext=(-38, 12),
171            textcoords="offset points")
172 ax.annotate(f"{dry4_s / 3600.0:.2f} h", (dry4_s / 3600.0, 0.15), xytext=(6, 12), textcoords="offset
173            points")
174 ax.set_xlabel("时间/h", ylabel="全域最大水分浓度/(kg/kg)", ylim=(0, None))
175 ax.legend(fontsize=8)
176 fig.tight_layout()
177 fig.savefig(FIGURE_DIR / "shrinkage_effect.png", dpi=180)
178 plt.close(fig)
179
180 fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8))
181 axes[0].plot(radius_history.time_s / 3600.0, radius_history.radius_m * 100.0)
182 axes[0].set_xlabel("时间/h", ylabel="半径/cm")
183 names = list(comparison_times_h)
184 vals = [comparison_times_h[name] for name in names]
185 axes[1].bar(range(len(vals)), vals, color=["#4C78A8", "#F58518", "#54A24B", "#B279A2"][: len(vals)
186 ])
187 axes[1].set_xticks(range(len(vals)), names, rotation=18, ha="right", fontsize=8)
188 axes[1].set_ylabel("烘干达标时间/h")
189 for idx, value in enumerate(vals):
190     axes[1].text(idx, value, f"{value:.2f}", ha="center", va="bottom", fontsize=8)
191 fig.tight_layout()
192 fig.savefig(FIGURE_DIR / "radius_and_comparison.png", dpi=180)
193 plt.close(fig)

```

A.10 四问计算流程：src/workflow.py

```
1 """四问求解、结果汇总、Excel 输出与绘图总流程。"""
2
3 import json
4 import math
5
6 import numpy as np
7
8 from .config import (
9     ATTACHMENT_DIR,
10     FIXED_COMPARISON_MAX_TIME_S,
11     INITIAL_RADIUS_M,
12     PLATEAU_START_S,
13     POST_DRY_S,
14     RADIUS_MAX_TIME_S,
15     RESULT_DIR,
16     TEMPLATE_DIR,
17 )
18 from .data_io import read_environment, read_radius
19 from .excel_output import write_single_sheet_result, write_two_sheet_result
20 from .plotting import create_figures
21 from .sampling import (
22     event_time,
23     first_node_output_time_below,
24     first_output_time_below,
25     grid_difference,
26     max_moisture,
27     representative_table,
28     sample_fixed,
29     sample_moving_moisture,
30 )
31 from .solver import solve_fixed, solve_moving
32
33 def run_all(write_excel: bool = True) -> dict:
34     RESULT_DIR.mkdir(parents=True, exist_ok=True)
35     environment = read_environment(ATTACHMENT_DIR / "附件1.xlsx")
36     radius_history = read_radius(ATTACHMENT_DIR / "附件2.xlsx")
37
38     print("[1/8] 求解问题一 (161 节点)", flush=True)
39     q1 = solve_fixed("问题一", 1800.0, 161, "appendix2", environment)
40     print("[2/8] 求解问题二 (201 节点)", flush=True)
41     q2 = solve_fixed("问题二", 10800.0, 201, "appendix3", environment)
42     print("[3/8] 求解问题三 (801 节点, 积分至 72 h)", flush=True)
43     q3 = solve_fixed("问题三", RADIUS_MAX_TIME_S, 801, "appendix3", environment, True)
44     dry3_s = event_time(q3)
45     safe3_s, safe3_c = first_output_time_below(q3)
46     visible3_s, visible3_c = first_output_time_below(q3, require_displayed_below=True)
47     print(
48         f"    问题三连续临界: {dry3_s / 3600:.6f} h; "
49         f"60 s网格首个严格达标: {safe3_s:.0f} s",
50         flush=True,
51     )
52     print("[4/8] 求解问题四 (601 节点, 含几何项)", flush=True)
53     q4 = solve_moving("问题四", RADIUS_MAX_TIME_S, 601, environment, radius_history, True, True)
54     dry4_s = event_time(q4)
```

```

55 safe4_s, safe4_c = first_output_time_below(q4)
56 visible4_s, visible4_c = first_output_time_below(q4, require_displayed_below=True)
57 print(
58     f"        问题四连续临界: {dry4_s / 3600:.6f} h; "
59     f"60 s网格首个严格达标: {safe4_s:.0f} s",
60     flush=True,
61 )
62
63 print("[5/8] 求解网格与模型敏感性对照", flush=True)
64 q1_coarse = solve_fixed("问题一粗网格", 1800.0, 81, "appendix2", environment)
65 q1_fine = solve_fixed("问题一细网格", 1800.0, 321, "appendix2", environment)
66 q2_coarse = solve_fixed("问题二粗网格", 10800.0, 101, "appendix3", environment)
67 q2_fine = solve_fixed("问题二细网格", 10800.0, 401, "appendix3", environment)
68 q3_coarse = solve_fixed("问题三粗网格", RADIUS_MAX_TIME_S, 401, "appendix3", environment, True)
69 q3_fine = solve_fixed("问题三细网格", RADIUS_MAX_TIME_S, 1201, "appendix3", environment, True)
70 q4_coarse = solve_moving("问题四粗网格", RADIUS_MAX_TIME_S, 321, environment, radius_history, True,
71     True)
72 q4_fine = solve_moving("问题四细网格", RADIUS_MAX_TIME_S, 801, environment, radius_history, True,
73     True)
74 q4_no_geometry = solve_moving("问题四无几何项", RADIUS_MAX_TIME_S, 601, environment, radius_history,
75     False, True)
76 appendix4_fixed = solve_fixed("附录4固定半径", FIXED_COMPARISON_MAX_TIME_S, 801, "appendix4",
77     environment, True)
78 dry3_coarse_s = event_time(q3_coarse)
79 dry3_fine_s = event_time(q3_fine)
80 dry4_coarse_s = event_time(q4_coarse)
81 dry4_fine_s = event_time(q4_fine)
82 dry4_no_geometry_s = event_time(q4_no_geometry)
83 dry_appendix4_fixed_s = event_time(appendix4_fixed)
84 safe_appendix4_fixed_s, safe_appendix4_fixed_c = first_output_time_below(appendix4_fixed)
85 q3_surface_safe_s, q3_surface_safe_c = first_node_output_time_below(q3, -1)
86 q4_radius_at_safe_m, _ = radius_history.value_and_rate(safe4_s)
87
88 output_r_cm = np.round(np.arange(0.0, 2.0 + 0.05, 0.1), 1)
89 output_r_m = output_r_cm / 100.0
90 if write_excel:
91     print("[6/8] 填写四个 Excel 结果模板", flush=True)
92     q1_times = np.arange(1.0, 1800.0 + 1.0, 1.0)
93     q1_t, q1_c = sample_fixed(q1, q1_times, output_r_m)
94     write_two_sheet_result(TEMPLATE_DIR / "result1.xlsx", q1_times, output_r_cm, q1_t, q1_c)
95
96     q2_times = np.arange(1.0, 10800.0 + 1.0, 1.0)
97     q2_t, q2_c = sample_fixed(q2, q2_times, output_r_m)
98     write_two_sheet_result(TEMPLATE_DIR / "result2.xlsx", q2_times, output_r_cm, q2_t, q2_c)
99
100     q3_end_s = min(RADIUS_MAX_TIME_S, safe3_s + POST_DRY_S)
101     q3_times = np.arange(60.0, q3_end_s + 0.1, 60.0)
102     _, q3_c = sample_fixed(q3, q3_times, output_r_m)
103     write_single_sheet_result(
104         TEMPLATE_DIR / "result3.xlsx", q3_times, output_r_cm.tolist(), q3_c
105     )
106
107     q4_end_s = min(RADIUS_MAX_TIME_S, safe4_s + POST_DRY_S)
108     q4_times = np.arange(60.0, q4_end_s + 0.1, 60.0)
109     q4_c, _ = sample_moving_moisture(q4, radius_history, q4_times, output_r_m)
110     q4_surface = q4.fields(q4_times)[1][-1, :]

```

```

107     q4_values = np.column_stack((q4_c, q4_surface))
108     q4_headers: list[float | str] = output_r_cm.tolist() + ["药材表面"]
109     write_single_sheet_result(
110         TEMPLATE_DIR / "result4.xlsx", q4_times, q4_headers, q4_values
111     )
112
113     print("[7/8] 汇总代表点、误差与可行性检查", flush=True)
114     q1_paper_times = np.asarray([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
115     paper_r_cm = np.asarray([0, 0.5, 1.0, 1.5, 2.0], dtype=float)
116     q1_pt, q1_pc = sample_fixed(q1, q1_paper_times, paper_r_cm / 100.0)
117     q2_paper = representative_table(q2, [0.5, 1, 1.5, 2, 2.5, 3], paper_r_cm.tolist())
118
119     q3_paper_hours = np.append(
120         np.arange(6.0, math.floor(safe3_s / 3600.0 / 6.0) * 6.0 + 0.1, 6.0),
121         safe3_s / 3600.0,
122     )
123     _, q3_paper_c = sample_fixed(q3, q3_paper_hours * 3600.0, paper_r_cm / 100.0)
124     q4_fixed_r_cm = np.asarray([0.0, 0.5, 1.0, 1.5], dtype=float)
125     q4_paper_hours = np.append(
126         np.arange(6.0, math.floor(safe4_s / 3600.0 / 6.0) * 6.0 + 0.1, 6.0),
127         safe4_s / 3600.0,
128     )
129     q4_paper_c, q4_paper_radii = sample_moving_moisture(
130         q4, radius_history, q4_paper_hours * 3600.0, q4_fixed_r_cm / 100.0
131     )
132     q4_paper_surface = q4.fields(q4_paper_hours * 3600.0)[1][-1, :]
133
134     def nullable_rows(array: np.ndarray) -> list[list[float | None]]:
135         return [
136             [None if not np.isfinite(value) else round(float(value), 4) for value in row]
137             for row in array
138         ]
139
140     validation_delta_s = 60.0
141     dry3_before = max_moisture(q3, max(0.0, dry3_s - validation_delta_s))
142     dry3_after = max_moisture(q3, min(RADIUS_MAX_TIME_S, dry3_s + POST_DRY_S))
143     dry4_before = max_moisture(q4, max(0.0, dry4_s - validation_delta_s))
144     dry4_after = max_moisture(q4, min(RADIUS_MAX_TIME_S, dry4_s + POST_DRY_S))
145     safe3_previous = max_moisture(q3, safe3_s - 60.0)
146     safe4_previous = max_moisture(q4, safe4_s - 60.0)
147
148     comparison_times_h = {
149         "附录3固定半径": dry3_s / 3600.0,
150         "附录4固定半径": dry_appendix4_fixed_s / 3600.0,
151         "附录4动态半径": dry4_s / 3600.0,
152         "动态半径无几何项": dry4_no_geometry_s / 3600.0,
153     }
154     summary = {
155         "environment": {
156             "plateau_start_s": PLATEAU_START_S,
157             "plateau_temperature_c": environment.plateau_temperature_k - 273.15,
158             "plateau_moisture": environment.plateau_moisture,
159         },
160         "radius": {
161             "initial_cm": radius_history.radius_m[0] * 100.0,
162             "at_72h_cm": radius_history.radius_m[-1] * 100.0,

```

```

163     },
164     "drying_time": {
165         "question3_continuous_threshold_s": dry3_s,
166         "question3_continuous_threshold_h": dry3_s / 3600.0,
167         "question3_safe_60s_output_s": safe3_s,
168         "question3_safe_60s_output_h": safe3_s / 3600.0,
169         "question3_first_displayed_below_s": visible3_s,
170         "question3_first_displayed_below_h": visible3_s / 3600.0,
171         "question4_continuous_threshold_s": dry4_s,
172         "question4_continuous_threshold_h": dry4_s / 3600.0,
173         "question4_safe_60s_output_s": safe4_s,
174         "question4_safe_60s_output_h": safe4_s / 3600.0,
175         "question4_first_displayed_below_s": visible4_s,
176         "question4_first_displayed_below_h": visible4_s / 3600.0,
177         "appendix4_fixed_h": dry_appendix4_fixed_s / 3600.0,
178         "question4_without_geometry_term_h": dry4_no_geometry_s / 3600.0,
179     },
180     "threshold_check": {
181         "question3_safe_60s_output_raw_max": safe3_c,
182         "question3_safe_60s_output_rounded_4dp": round(safe3_c, 4),
183         "question3_previous_60s_output_raw_max": safe3_previous,
184         "question3_first_displayed_below_raw_max": visible3_c,
185         "question3_60s_before": dry3_before,
186         "question3_30min_after": dry3_after,
187         "question4_safe_60s_output_raw_max": safe4_c,
188         "question4_safe_60s_output_rounded_4dp": round(safe4_c, 4),
189         "question4_previous_60s_output_raw_max": safe4_previous,
190         "question4_first_displayed_below_raw_max": visible4_c,
191         "question4_60s_before": dry4_before,
192         "question4_30min_after": dry4_after,
193     },
194     "abstract_metrics": {
195         "question1_at_1800s": {
196             "center_temperature_c": float(q1_pt[-1, 0]),
197             "surface_temperature_c": float(q1_pt[-1, -1]),
198             "center_moisture": float(q1_pc[-1, 0]),
199             "surface_moisture": float(q1_pc[-1, -1]),
200         },
201         "question2_at_3h": {
202             "center_temperature_c": float(q2_paper["temperature_c"][-1][0]),
203             "surface_temperature_c": float(q2_paper["temperature_c"][-1][-1]),
204             "center_moisture": float(q2_paper["moisture"][-1][0]),
205             "surface_moisture": float(q2_paper["moisture"][-1][-1]),
206         },
207         "question3_surface_safe_60s_s": q3_surface_safe_s,
208         "question3_surface_safe_60s_h": q3_surface_safe_s / 3600.0,
209         "question3_surface_safe_raw_moisture": q3_surface_safe_c,
210         "question3_center_to_surface_drying_time_ratio": safe3_s / q3_surface_safe_s,
211         "question3_center_minus_surface_drying_time_h": (safe3_s - q3_surface_safe_s) / 3600.0,
212         "question3_center_to_surface_moisture_ratio_at_finish": safe3_c / float(q3.fields(safe3_s)
213 [1][-1]),
214         "question4_radius_at_finish_cm": q4_radius_at_safe_m * 100.0,
215         "appendix4_fixed_safe_60s_s": safe_appendix4_fixed_s,
216         "appendix4_fixed_safe_60s_h": safe_appendix4_fixed_s / 3600.0,
217         "appendix4_fixed_safe_raw_moisture": safe_appendix4_fixed_c,
218         "question4_time_reduction_vs_fixed_h": (safe_appendix4_fixed_s - safe4_s) / 3600.0,

```

```

218         "question4_time_reduction_vs_fixed_percent": (safe_appendix4_fixed_s - safe4_s) /
safe_appendix4_fixed_s * 100.0,
219         "question4_drying_speed_ratio_vs_fixed": safe_appendix4_fixed_s / safe4_s,
220     },
221     "grid_convergence": {
222         "question1_max_temperature_difference_81_vs_321_at_1800s_K": grid_difference(q1_coarse,
q1_fine, 1800.0, "temperature"),
223         "question1_max_moisture_difference_81_vs_321_at_1800s": grid_difference(q1_coarse, q1_fine,
1800.0, "moisture"),
224         "question1_max_temperature_difference_161_vs_321_at_1800s_K": grid_difference(q1, q1_fine,
1800.0, "temperature"),
225         "question1_max_moisture_difference_161_vs_321_at_1800s": grid_difference(q1, q1_fine,
1800.0, "moisture"),
226         "question2_max_temperature_difference_101_vs_401_at_3h_K": grid_difference(q2_coarse,
q2_fine, 10800.0, "temperature"),
227         "question2_max_moisture_difference_101_vs_401_at_3h": grid_difference(q2_coarse, q2_fine,
10800.0, "moisture"),
228         "question2_max_temperature_difference_201_vs_401_at_3h_K": grid_difference(q2, q2_fine,
10800.0, "temperature"),
229         "question2_max_moisture_difference_201_vs_401_at_3h": grid_difference(q2, q2_fine, 10800.0,
"moisture"),
230         "question3_dry_time_h_401_nodes": dry3_coarse_s / 3600.0,
231         "question3_dry_time_h_801_nodes": dry3_s / 3600.0,
232         "question3_dry_time_h_1201_nodes": dry3_fine_s / 3600.0,
233         "question4_dry_time_h_321_nodes": dry4_coarse_s / 3600.0,
234         "question4_dry_time_h_601_nodes": dry4_s / 3600.0,
235         "question4_dry_time_h_801_nodes": dry4_fine_s / 3600.0,
236     },
237     "paper_tables": {
238         "question1": {
239             "times_s": q1_paper_times.astype(int).tolist(),
240             "distances_cm": paper_r_cm.tolist(),
241             "temperature_c": np.round(q1_pt, 4).tolist(),
242             "moisture": np.round(q1_pc, 4).tolist(),
243         },
244         "question2": q2_paper,
245         "question3": {
246             "times_h": np.round(q3_paper_hours, 6).tolist(),
247             "distances_cm": paper_r_cm.tolist(),
248             "moisture": np.round(q3_paper_c, 4).tolist(),
249         },
250         "question4": {
251             "times_h": np.round(q4_paper_hours, 6).tolist(),
252             "fixed_distances_cm": q4_fixed_r_cm.tolist(),
253             "radius_cm": np.round(q4_paper_radii * 100.0, 4).tolist(),
254             "moisture_at_fixed_distances": nullable_rows(q4_paper_c),
255             "surface_moisture": np.round(q4_paper_surface, 4).tolist(),
256         },
257     },
258     "solver_stats": {
259         sim.name: {"nfev": int(sim.solution.nfev), "njev": int(sim.solution.njev)}
260         for sim in [q1, q2, q3, q4]
261     },
262 }
263 (RESULT_DIR / "summary.json").write_text(
264     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"

```

```

265     )
266
267     print("[8/8] 生成说明书图表", flush=True)
268     create_figures(
269         environment,
270         radius_history,
271         q1,
272         q2,
273         q3,
274         q4,
275         appendix4_fixed,
276         dry3_s,
277         dry4_s,
278         safe3_s,
279         safe4_s,
280         comparison_times_h,
281     )
282     print("全部数值任务完成。", flush=True)
283     return summary

```

A.11 参数敏感性分析: src/sensitivity_analysis.py

```

1  """关键传热传质参数的正负10%敏感性分析。"""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  from . import physics, solver
9  from .config import ATTACHMENT_DIR, RADIUS_MAX_TIME_S
10 from .data_io import read_environment, read_radius
11 from .sampling import event_time, first_output_time_below
12
13
14 OUTPUT_PATH = Path(__file__).resolve().parents[1] / "result" / "parameter_sensitivity.json"
15
16
17 def one_case(parameter: str, factor: float) -> dict[str, float]:
18     original_h = physics.HEAT_TRANSFER
19     original_hm = physics.MASS_TRANSFER
20     original_d = physics.DIFFUSIVITY_SCALE
21     try:
22         if parameter == "heat_transfer_h":
23             physics.HEAT_TRANSFER = original_h * factor
24         elif parameter == "mass_transfer_hm":
25             physics.MASS_TRANSFER = original_hm * factor
26         elif parameter == "diffusivity_scale":
27             physics.DIFFUSIVITY_SCALE = original_d * factor
28         elif parameter != "baseline":
29             raise ValueError(parameter)
30
31     environment = read_environment(ATTACHMENT_DIR / "附件1.xlsx")
32     radius = read_radius(ATTACHMENT_DIR / "附件2.xlsx")

```

```

33     q3 = solver.solve_fixed(
34         f"问题三-{{parameter}}-{{factor}}",
35         RADIUS_MAX_TIME_S,
36         801,
37         "appendix3",
38         environment,
39         True,
40     )
41     q4 = solver.solve_moving(
42         f"问题四-{{parameter}}-{{factor}}",
43         RADIUS_MAX_TIME_S,
44         601,
45         environment,
46         radius,
47         True,
48         True,
49     )
50     event3 = event_time(q3)
51     event4 = event_time(q4)
52     safe3, _ = first_output_time_below(q3)
53     safe4, _ = first_output_time_below(q4)
54     return {
55         "question3_continuous_h": event3 / 3600.0,
56         "question3_safe_60s_h": safe3 / 3600.0,
57         "question4_continuous_h": event4 / 3600.0,
58         "question4_safe_60s_h": safe4 / 3600.0,
59     }
60 finally:
61     physics.HEAT_TRANSFER = original_h
62     physics.MASS_TRANSFER = original_hm
63     physics.DIFFUSIVITY_SCALE = original_d
64
65
66 def run_sensitivity(print_output: bool = True) -> dict:
67     baseline = one_case("baseline", 1.0)
68     cases: dict[str, dict[str, dict[str, float]]] = {}
69     for parameter in ["heat_transfer_h", "mass_transfer_hm", "diffusivity_scale"]:
70         cases[parameter] = {
71             "minus_10_percent": one_case(parameter, 0.9),
72             "plus_10_percent": one_case(parameter, 1.1),
73         }
74
75     for parameter, variants in cases.items():
76         for variant, values in variants.items():
77             for question in ["question3", "question4"]:
78                 base_h = baseline[f"{{question}}_continuous_h"]
79                 changed_h = values[f"{{question}}_continuous_h"]
80                 values[f"{{question}}_relative_change_percent"] = (
81                     (changed_h - base_h) / base_h * 100.0
82                 )
83
84     output = {
85         "perturbation": "each parameter independently multiplied by 0.9 or 1.1",
86         "baseline_parameters": {
87             "heat_transfer_h_W_m2K": physics.HEAT_TRANSFER,
88             "mass_transfer_hm_m_s": physics.MASS_TRANSFER,

```

```

89         "diffusivity_scale": physics.DIFFUSIVITY_SCALE,
90     },
91     "baseline": baseline,
92     "cases": cases,
93 }
94 OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)
95 OUTPUT_PATH.write_text(json.dumps(output, ensure_ascii=False, indent=2), encoding="utf-8")
96 if print_output:
97     print(json.dumps(output, ensure_ascii=False, indent=2))
98 return output
99
100
101 def main() -> None:
102     run_sensitivity(print_output=True)
103
104
105 if __name__ == "__main__":
106     main()

```

A.12 兼容调用入口: src/drying_solver.py

```

1  r"""A题 “药材的烘干问题” 数值求解兼容入口。
2
3  核心实现已按用途拆分到 config、models、data_io、physics、solver、
4  sampling、excel_output、plotting 和 workflow 模块。建议从支撑材料
5  根目录运行 python run_all.py。
6  """
7
8  from __future__ import annotations
9
10 import argparse
11
12 from .config import (
13     ATTACHMENT_DIR,
14     DATA_DIR,
15     FIGURE_DIR,
16     FIXED_COMPARISON_MAX_TIME_S,
17     INITIAL_C,
18     INITIAL_RADIUS_M,
19     INITIAL_T_K,
20     PLATEAU_START_S,
21     POST_DRY_S,
22     PROJECT_ROOT,
23     RADIUS_MAX_TIME_S,
24     RESULT_DIR,
25     TEMPLATE_DIR,
26 )
27 from .data_io import read_environment, read_radius
28 from .models import Environment, RadiusHistory, Simulation
29 from .physics import material_properties
30 from .sampling import (
31     event_time,
32     first_node_output_time_below,
33     first_output_time_below,

```

```

34     grid_difference,
35     max_moisture,
36     representative_table,
37     sample_fixed,
38     sample_moving_moisture,
39 )
40 from .solver import (
41     coupled_jacobian_sparsity,
42     dry_event,
43     make_fixed_rhs,
44     make_moving_rhs,
45     solve_fixed,
46     solve_moving,
47 )
48 from .workflow import run_all
49
50
51 def main() -> None:
52     parser = argparse.ArgumentParser(description=__doc__)
53     parser.add_argument(
54         "--no-excel",
55         action="store_true",
56         help="只计算和绘图，不改写附件3结果文件",
57     )
58     args = parser.parse_args()
59     run_all(write_excel=not args.no_excel)
60
61
62 if __name__ == "__main__":
63     main()

```

A.13 源码包声明：src/__init__.py

```

1  """A题药材烘干数值计算包。"""
2
3  from .workflow import run_all
4
5  __all__ = ["run_all"]

```

A.14 结果完整性校验：validate_results.py

```

1  """校验支撑材料的结果文件、时间边界与图表完整性。"""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  from pathlib import Path
8
9  from openpyxl import load_workbook
10
11

```

```

12 ROOT = Path(__file__).resolve().parent
13 RESULT_DIR = ROOT / "result"
14 FIGURE_DIR = ROOT / "report" / "figures"
15
16 EXPECTED_WORKBOOKS = {
17     "result1.xlsx": {"sheets": ["温度", "水分浓度"], "rows": 1801, "cols": 22, "first_time": 1, "
18         last_time": 1800},
19     "result2.xlsx": {"sheets": ["温度", "水分浓度"], "rows": 10801, "cols": 22, "first_time": 1, "
20         last_time": 10800},
21     "result3.xlsx": {"sheets": ["Sheet1"], "rows": 3486, "cols": 22, "first_time": 60, "last_time":
22         209100},
23     "result4.xlsx": {"sheets": ["Sheet1"], "rows": 3194, "cols": 23, "first_time": 60, "last_time":
24         191580},
25 }
26
27 EXPECTED_DISTANCE_HEADERS = [round(i / 10, 1) for i in range(21)]
28
29 EXPECTED_FIGURES = [
30     "drying_threshold.png",
31     "environment_extension.png",
32     "question1_heatmaps.png",
33     "question1_profiles.png",
34     "question2_heatmaps.png",
35     "question2_profiles.png",
36     "question3_heatmaps.png",
37     "question4_heatmaps.png",
38     "radius_and_comparison.png",
39     "shrinkage_effect.png",
40 ]
41
42 def check(condition: bool, message: str) -> None:
43     if not condition:
44         raise AssertionError(message)
45
46 def validate_workbook(name: str, spec: dict) -> dict:
47     path = RESULT_DIR / name
48     check(path.is_file(), f"缺少结果文件: {path}")
49     wb = load_workbook(path, read_only=True, data_only=True)
50     try:
51         check(wb.sheetnames == spec["sheets"], f"{name} 工作表名称或顺序不符: {wb.sheetnames}")
52         for ws in wb.worksheets:
53             check(ws.max_row == spec["rows"], f"{name}/{ws.title} 行数应为 {spec['rows']}, 实际为 {ws.
54                 max_row}")
55             check(ws.max_column == spec["cols"], f"{name}/{ws.title} 列数应为 {spec['cols']}, 实际为 {ws.
56                 max_column}")
57             check(
58                 ws.cell(1, 1).value == "时间\\到药材中心的距离",
59                 f"{name}/{ws.title} A1 表头错误",
60             )
61             distance_headers = [ws.cell(1, col).value for col in range(2, 23)]
62             check(distance_headers == EXPECTED_DISTANCE_HEADERS, f"{name}/{ws.title} 距离表头错误")
63             if name == "result4.xlsx":
64                 check(ws.cell(1, 23).value == "药材表面", "result4.xlsx 缺少药材表面列")
65             check(ws.cell(2, 1).value == spec["first_time"], f"{name}/{ws.title} 首个时间点错误")

```

```

62         check(ws.cell(ws.max_row, 1).value == spec["last_time"], f"{name}/{ws.title} 末个时间点错误"
63     )
64     return {"status": "pass", "sheets": wb.sheetnames, "rows": spec["rows"], "cols": spec["cols"]}
65 finally:
66     wb.close()
67
68 def run_validation(require_sensitivity: bool = True) -> dict:
69     summary_path = RESULT_DIR / "summary.json"
70     check(summary_path.is_file(), "缺少 result/summary.json")
71     summary = json.loads(summary_path.read_text(encoding="utf-8"))
72     threshold = summary["threshold_check"]
73
74     check(threshold["question3_safe_60s_output_raw_max"] < 0.15, "问题三报告时点未严格低于 0.15")
75     check(threshold["question3_previous_60s_output_raw_max"] >= 0.15, "问题三报告时点不是首个 60 s 严格达
76         标点")
77     check(threshold["question4_safe_60s_output_raw_max"] < 0.15, "问题四报告时点未严格低于 0.15")
78     check(threshold["question4_previous_60s_output_raw_max"] >= 0.15, "问题四报告时点不是首个 60 s 严格达
79         标点")
80
81     workbook_checks = {name: validate_workbook(name, spec) for name, spec in EXPECTED_WORKBOOKS.items()
82     }
83     missing_figures = [name for name in EXPECTED_FIGURES if not (FIGURE_DIR / name).is_file()]
84     check(not missing_figures, f"缺少图表: {missing_figures}")
85
86     sensitivity_path = RESULT_DIR / "parameter_sensitivity.json"
87     if require_sensitivity:
88         check(sensitivity_path.is_file(), "缺少 result/parameter_sensitivity.json")
89
90     report = {
91         "status": "pass",
92         "workbooks": workbook_checks,
93         "strict_threshold": {
94             "question3_time_s": summary["drying_time"]["question3_safe_60s_output_s"],
95             "question3_raw_max": threshold["question3_safe_60s_output_raw_max"],
96             "question4_time_s": summary["drying_time"]["question4_safe_60s_output_s"],
97             "question4_raw_max": threshold["question4_safe_60s_output_raw_max"],
98         },
99         "figures": {"count": len(EXPECTED_FIGURES), "status": "pass"},
100         "sensitivity": "pass" if sensitivity_path.is_file() else "skipped",
101     }
102     (RESULT_DIR / "validation.json").write_text(
103         json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
104     )
105     return report
106
107 def main(require_sensitivity: bool | None = None) -> None:
108     if require_sensitivity is None:
109         parser = argparse.ArgumentParser(description=__doc__)
110         parser.add_argument("--allow-missing-sensitivity", action="store_true")
111         args = parser.parse_args()
112         require_sensitivity = not args.allow_missing_sensitivity
113     report = run_validation(require_sensitivity=require_sensitivity)
114     print(json.dumps(report, ensure_ascii=False, indent=2))

```

```
114
115 if __name__ == "__main__":
116     main()
```